PHYC40870
SPACE DETECTOR LABORATORY

MSC SPACE SCIENCE

AUTHORS:
MORGAN FRASER (MORGAN.FRASER@UCD.IE, UCD SCHOOL OF PHYSICS)
ANDREW KIRWAN (ANDREW.KIRWAN1@UCD.IE, UCD SCHOOL OF PHYSICS)
ROBERT JEFFREY (UNIVERSITY OF SOUTHAMPTON SCHOOL OF PHYSICS & ASTRONOMY)

VERSION 0.1, 28 MAY 2025
VERSION 1.0, 7 SEPT 2025
VERSION 2.0, 25 SEPT 2025
VERSION 3.0, 12 NOV 2025

Contents

Introduction to PHYC40870

Module Overview

The Space Detector Laboratory is a 10 credit core module for the MSc Space Science & Technology course. This is a lab-based module with lab sessions scheduled for Mondays and Tuesdays from 14:00-17:00. You will be expected to do independent work, including (but not limited to) preparation, reading, coding, and report writing outside of these hours.

This module is intended to instill a working, functional understanding of detector systems and imaging technology, as well as provide practical experience with "CubeSat" and nano-satellite systems. These skills will prepare you for both the Space Mission Design field trip, and the Satellite Subsystems lab in Semester 2. Additionally, we will introduce you to some essential coding skills in the Python programming language, which will further aid you later in your studies.

The module has three primary components:

- Fundamentals of Python
- CCDs & Imaging Techniques
- Gamma-ray Detectors in Space & Astronomy
- "AMSAT" (**Am**ateur **Sat**ellite) and CubeSat systems

Course schedule

W1	Mon 8th Sept	Introduction to Python
	Tue 9th	Introduction to Python
W2	Mon 15th	Introduction to Python
	Tues 16th	GitHub
W3	Mon 22nd	Virtual environments
	Tues 23rd	ChatGPT
W4	Mon 29th	Optical detectors lecture
	Tues 30th	HST data exercise
W5	Mon 6th Oct	HST data exercise
	Tues 7th	Sentinel-2 exercise
W6	Mon 13th	Sentinel-2 exercise
	Tues 14th	High energy detectors lecture
W7	Mon 20th	Detector 1 experiment
	Tues 21st	Detector 1 experiment
W8	Mon 27th	Public holiday
	Tues 28th	Detector 1 experiment
W9	Mon 3rd Nov	Detector 2 experiment
	Tues 4th	Detector 2 experiment
W10	Mon 10th	Detector 2 experiment
	Tues 11st	Merge detector results
W11	Mon 17th	AMSAT - introduction
	Tues 18th	AMSAT - connecting
W12	Mon 24th	AMSAT experiments
	Tues 25th	AMSAT experiments

Assessment & Deliverables

Throughout the course you will be assessed through a combination of assignments and continuous assessments. The assessed deliverables are as follows:

1. Programming in Python
   - Code submissions (20%)
2. Optical and Other Detectors
   - Code submissions (25%)
3. Gamma-ray Detectors in Astronomy
   - Code submissions (15%)
   - Final Report (15%)
4. AMSAT
   - Final Report (25%)

In some cases you may be asked to put some of your work on a GitHub repository, however *in all cases you will need to also upload these onto Brightspace*

Computing fundamentals

Python

Why Python?

Python is an open-source, easily accessible, and powerful programming and scripting language which has become increasingly popular in a variety of disciplines, particularly astronomy and physics. The most accessible and practical use of the language is with the Raspberry Pi computer and other microcontrollers, as these are especially well-documented by DIY enthusiasts, "makers", and electronics hobbyists. We see it used in higher-level situations as well: for example, CubeSats and nano-satellites often have software/hardware interfaces controlled with the Python language, and state-of-the-art telescopes like the James Webb Space Telescope (JWST) feature data reduction pipelines written almost entirely in Python.

Perhaps most attractive is the simplicity and human-like syntax of the language; this ease of use has made Python a choice language for statistics, machine learning, and data reduction and visualization. Additionally, its popularity has led to the community development of countless tools (called "modules" or packages) that can be imported into your workspace and used with minimal effort, preventing the need to "reinvent the wheel" when writing your own code for data analysis. Overall it is a user-friendly and easy-to-learn language that offers accessibility to a host of disciplines for both the humble beginner and the seasoned programmer.

Some of you may have prior experience with Python, though we anticipate that few of you will have experience using it to write more extended software or for interfacing with hardware. In our practical sessions, we will be helping to lay a foundation for your learning in this respect in order to prepare you not only for upcoming modules, but for your future careers as well. To this end we have provided a number of resources for you to utilize throughout the course; as our laboratory time is limited, it is expected that you consult these references outside of our scheduled times, and familiarize yourself

with some of the fundamentals prior to our sessions. For ease of reference you can find these supplementary sources at the end of this section.

Installation

There are several ways to install Python, and the method you choose really depends on what you are most comfortable doing. The easiest method by far is through the Anaconda distribution platform, which has two "flavors" or installations from which to choose: miniconda and anaconda. Both of these version rely on the same underlying architecture which installs both Python and the command-line tool `conda`. Miniconda is a lightweight version which only installs the bare minimum (that is, *only* Python and `conda`), while Anaconda includes a massive collection of pre-installed packages that are commonly used in scientific computing.

For this course we recommend the 'anaconda' package. Full details of how to use the architecture can be found on their website, and you have the option to create an account or skip registration; it is easiest to simply skip registration and proceed to the download. Once it is installed, you can access the Anaconda Navigator and open a Jupyter Notebook to get started. You should spend some time getting to know the Navigator.

If you feel adventurous and want to use the command line, you can access the command prompt by using the Windows Start Menu (if you are using Windows), or find it within the Anaconda Navigator. On MacOS or Linux, you can open up a terminal and type `conda activate`. Once you've activated miniconda you should see something like

```
(base) C:\Users\YourUsername>    # for Windows
[...]
(base) user@hostname:~$         # for Mac/Linux
```

Throughout this handbook, we will present all operations from the command line or terminal in the style:

```
$ [code goes here]
```

Virtual environments

By default, miniconda will activate the "base" environment so you might see `(base)` at the beginning of the command line. Environments are a useful way to keep your installed packages compartmentalized. This is especially important if you're designing some kind of software that other people might want to download and install, as

it ensures that all the necessary *dependencies* (the packages that your project depends on to work) are satisfied within that project environment. It also prevents you from accidentally removing a package that your computer might need in order to run, or from adding a different version of a package that might conflict with a similar package on your computer. While the Anaconda ecosystem is generally quite good at handling dependency conflicts, it's still best practice to create a *virtual environment* for your project to keep things clean.

To create an environment, you can simply open the command terminal and type

```
$ conda create -n NAME
```

The `conda` commands `activate NAME` and `deactivate NAME` will either activate or deactivate an environment you have named `NAME`. You can install packages using the `install` command. For our course, the most useful packages are `numpy`, `scipy`, `matplotlib`, and `jupyter`. We will discuss packages in more depth shortly.

> Coding Exercise:
> Use the `conda` package to create a virtual environment called `spacelab` and install the four packages listed above.

Python Basics

Language Structure

As discussed above, a stand-out feature of Python is its *syntax* or structure. Since Python was designed as a teaching language, it was important early on to have a structure that is simple and natural. To demonstrate this, let us a consider a simple problem.

We want to write a code that takes a list of numbers, checks each number in the list to see if it is even or odd, and makes a new list of the squares of only the even numbers. No matter the programming language we choose, the first step we should always take is to plan out our goals. What is the problem? What steps do we need to take to solve it? How do we put it all together to test it? One helpful method is to use "pseudocode", which is essentially just an informal, plain-language description of your algorithm. For our problem, we could have the following pseudocode:

```
PROCEDURE: find even numbers in a list
    and return their squares
CREATE empty list called result
READ list of numbers
FOR each number in numbers
```

```
    IF number is even, THEN
        square = number x number
        APPEND square to result
    ELSE number is odd, then
        PRINT the number is an odd number
RETURN result
```

This uses (or tries to use) plain language to describe the steps being taken to accomplish the goal, where the we have capitalized keywords to emphasize that they are commands we want to give to the program. We have written the Python equivalent of this below.

```
# create an empty list
result = []
# make a list of numbers
numbers = [1, 2, 3, 4, 5, 10, 13, 15, 18]

# check if the numbers are even or odd
for number in numbers:
    if number % 2 == 0:
        square = number**2
        result.append(number)
    else:
        print(f"{number} is not even.")

# print the result
print("The squared numbers are:")
print(result)
```

This is remarkably similar to the pseudocode. Before we go further it is important that you notice a few things right now.

- **Comments are useful**. Any text that is preceded by # will be treated as a comment and ignored by Python. Use them to document your code so when other programmers read it they will be able to quickly understand what you are doing.
- **Indentation matters**. Code within `for`, `if`, and `else` statements is always indented; you can press `tab` on your keyboard once, or the `space` key four times to do this. It also makes the code easier to read. Python will also expect an indentation after a colon (`:`).
- **Colons are important**. We put them at the end of a line of code if we intend to write a statement (such as a loop) or define a function, in which case it is always followed by an indentation. We can also use colons to access element of lists and perform other powerful operations on data structures (we will get to this later).

We also see some typical Python workings here as well: iterating over a list using a `for`-loop, controlling the logic or flow of the code with `if-else` statements; mathematical operators like modulo division (`%`) and exponents (`**`), adding new items to a list (`.append()`), and so-called "f-strings" which help format things for displaying to the user. And perhaps most importantly, it is easy to read. Compare the above code with its equivalent in the elder tongue of FORTRAN:

```
program square_evens
   implicit none
   integer, dimension(9) :: numbers = [1, 2, 3, 4, 5, 10, 13, 15, 18]
   integer, dimension(9) :: result
   integer :: i, count, square

   count = 0

   do i = 1, size(numbers)
      if (mod(numbers(i), 2) == 0) then
         square = numbers(i) ** 2
         count = count + 1
         result(count) = square
      else
         print *, numbers(i), "is not even."
      end if
   end do

   print *, "The squared numbers are:"
   do i = 1, count
      print *, result(i)
   end do
end program square_evens
```

There is quite a bit going on here, perhaps most important being that we have to explicitly define the "type" of the variables we are using. In Python we can simply define `numbers = [1, 2, 3]` and Python knows this is a list of integer values and can implicitly figure out how long the list is. In FORTRAN we have to "declare" that `numbers` is a list of integers *and* we have to tell FORTRAN how long it is (9 elements in this case). There is some similarity though with the `if` and `else` blocks and hierarchical indentation, and we can also see that we can end a line of code with whitespace (that is, just hit `enter` and move to the next line), but even the most simple code can sometimes feel inscrutable. FORTRAN is an old language though; what if we saw a more modern example, like C++?[1]

[1] Interestingly, Python modules like `NumPy` primarily use the C language, and `NumPy` specifically actually calls FORTRAN software packages to do much of its numerical heavy-lifting (e.g., linear algebra and matrix operations).

```c
#include <stdio.h>

int main() {
    int numbers[] = {1, 2, 3, 4, 5, 10, 13, 15, 18};
        int numSize = sizeof(numbers) / sizeof(numbers[0]);
    int result[numSize];
    int count = 0;
    int i, square;

    for (i = 0; i < numSize; i++) {
        if (numbers[i] % 2 == 0) {
            square = numbers[i] * numbers[i];
            result[count] = square;
            count++;
        }
        else {
            printf("%d is not even.\n", numbers[i]);
        }
    }
    printf("The squared numbers are: ");
    for (i = 0; i < count; i++) {
        printf("%d ", result[i]);
    }
    printf("\n");
    return 0;
}
```

Once again we see that we have to define the types of some of our variables, which we do not need to do in Python. We also have to end statements with a semi-colon, and we have to be more explicit than FORTRAN in our `for`-loop by telling it exactly how many numbers to use, e.g. `i = 0; i < numSize; i++` tells it to start the loop at 0 and increment each pass by 1 until `i` is less than the size of `numbers`. Perhaps it is easy to see why we are interested in Python, particularly if we are not intending to become computer scientists![2]

[2] A strong argument can be made, however, that learning a "statically typed" language can deepen your understanding of Python, much like learning (or already knowing!) another language can deepen your understanding of English.

Variables & Types

In the Python code above, we waved away a few explanations so you could look solely at the structure of the language, particularly as compared to a few other programming languages. Python is considered a "dynamically typed" language, meaning that that we are not required to explicitly declare the "type" of a variable before using: Python just accepts that you are creating a variable and does its best

to work with that. By **variable** we mean a placeholder where we have assigned some value for Python to store in memory. This assignment is simply done with a single = sign, e.g. `y=7`. We can perform operations on variables, re-assign new values to (some of) them, and otherwise manipulate them for all our computing needs.

Even though you are not required to declare the type of the variable, it is still helpful to understand data types in Python: for example if you try to perform a mathematical operation such as division between a `string`-type variable and a `integer`-type variable, Python will certainly complain because it is the proverbial apples and oranges. We can always check the type of our variable if we need to using the in-built `type()` method, for example `type(y)`. Understanding data types will become especially important in later modules, so we should get some familiarity with them as soon as we can.

Python has quite a few built-in data types, but the most common we will be using are:

- **Numeric types** (integers, floats, and complex numbers);
- **Text types** (called strings);
- **Boolean types**: (like `True` and `False`);
- **Sequence types**: (lists, sets, and ranges, collectively called "iterables");
- **Mapping types** (dictionaries and collections);
- **Null type** (special case representing emptiness);
- **Binary types** (bytes)

A full list of built-in types and the operations we can use on them can be found on the Python Standard Library documentation page.

Expressions, Operators, & Controlling the Flow

The behavior of standard mathematical operators should be quite predictable for you. We can further use the relationships defined by some mathematical symbols to create **relational** or **comparison operators**, which will return a `True` or `False` value. In Python, our relational operators are[3]
Note that the order is important here; `<=` will work but `=<` will raise an error.

We may also want to use the logical operators `and`, `or`, and `not`.[4] These correspond to the logical operations of **conjunction**, **disjunction**, and **negation**, respectively, and can be thought of as a type of binary operation. For example, if we define `x = 10`, we can

[3] A companion to the standard logical operators are "bitwise" operators", but we will not spend time on these here. You can find a few exercises with these operators in the bonus Jupyter notebook.

[4] If you are rusty, it may be worthwhile to refresh your memory regarding Truth tables and logical operations.

`x == y`	x is equal to y
`x != y`	x is not equal to y
`x > y`	x is greater than y
`x >= y`	x is greater than or equal to y
`x < y`	x is less than y
`x <= y`	x is less than or equal to y

evaluate statements like `(x < 1) and (x > 3)` (which is false) or `(x > 5) and (x % 5 == 0)` (which is true).

This is where the power of programming truly shines. By controlling the flow of our code, we can fine-tune how the code behaves under specified conditions and create programs as simple or sophisticated as we like. We are mainly interested here in **conditional statements**, **loop statements**, and **exceptions**. Whatever statement we are using, it is important to note that each statement has a header and a body, where the header is terminated with a colon (`:`) and the body contains the statements we want to execute based on the evaluation of the header.

You have seen **conditional statements** already with the `if-else` syntax. As the name implies, these are boolean expressions tell Python to only execute some block of code only if certain conditions are met. The simplest form is the `if` statement, which will only make one thing happen if the condition is true; if we want something else to happen if the condition is false, we can use `else`. We can use "chained conditionals" for cases where there are several possibilities we want to capture, in which case we through in an `elif` statement between `if` and `else` as many times as we need.

```
if statement_one:
    # do something
elif statement_two:
    # do something else
else:
    # do something different
```

Loop statements are most commonly seen as `for` and `while` blocks. Above, our code looped through the elements or values of a list, which as you recall are *iterable* data types in Python. If our iterable is a `dict` type, we have to specify the key as well as the value and use `.items()`:

```
for key, value in some_dictionary.items():
    # do something
```

Additionally, if we want to keep track of the index of each value in the loop, we can use the built-in `enumerate`:

```
for i, num in enumerate(numbers):
    print(f"Index: {i} \t Number: {num}")
```

The `while`-loop is also incredibly useful, as it allows us to execute a block of code while the statement is `True` and exit the code when the statement is `False`. Take care when using this, however, as an incorrectly written `while` loop could lead to an infinite loop. A simple usage may be

```
i = 1
while i < 10:
    print(f"Loop: {i}")
    i += 1
```

This simply prints the variable `i` so long as `i < 10`, incrementing the value of `i` in each loop. When the value reaches 10, the loop exits.

When using any type of loop, three common ways to fine-tune control of the loop are the `break`, `continue`, and `pass` statements. We use `break` when a condition has been met to gracefully exit the loop. The `continue` statement, on the other hand, tells the program that it can skip its current loop and continue with the rest of its processes. For example, if we want to immediately exit a `for` loop when a number is odd, we might have

```
for number in numbers:
    if number % 2 != 0:
        print(number)
        break
```

and the entire loop will end. If instead we want to skip the odd numbers and only print the even numbers, we could use

```
for number in number:
    if number % 2 == 0:
        print(number")
        continue
```

The `pass` statement can sometimes be easy to confuse with `continue` but it will produce very different results. We use `pass` as a placeholder for empty statements, as it tells the code to do nothing. We will talk more about this in our workshops.

Input/Output Streams

You will come across situations where you need to handling input and output processes: for example, if your program asks a user to provide some kind of input, or if you want to read data from a file or write data to a file. In the most simple case, we can accept input from a user with the `input()` function, which will store the entry as

a `str` regardless of what is provided. I/O streams can be text-type, binary-type, or raw-type, and Python has functionality for handling all three of these. For this course the most common type will be text I/O, specifically from reading text files. It is important to note here that **all files are text files**, although that text may just be stored a bit differently!

The standard way to handle file streams (that is, simply reading a file line by line) is with the `open()` function:

```
data = open(filename, mode='r')
```

where our `mode` tells the file reader how we want to handle the file. If we only want to read the file, we can simply use `r`. If we want to update a file, we can use `a` (append) or `r+` (read and append). We may be tempted to use `w` to write to the file, but beware that this truncates the file before writing: that is, it overwrites the existing data. By default, Python understands that you want to read a text file, but you can also specify `b` if your data is in a binary format, e.g. `open(filename, 'rb')`.

To actually read the data in the file, we have a few options. The `readline` method will do what it says: read a line. Note that this will read one line at a time, stopping at a "newline" or `\n` character; but the `file` object we created with `open` keeps track of where it left off, so we can keep calling `data.readline()` to print the next lines in the file. We can read all of the lines at once with `readlines`, which stores every line as an element in a list, or iterate over the lines in the `file` object with a `for`-loop. When we are done with the file it is important to close the file, e.g. `data.close()`.

A better way to handle file reading is by using the `with` statement and reading line by line:

```
with open(filename, 'r') as data:
    for line in data.readlines():
        # do some code here
```

This saves us from accidentally forgetting to close the file, as the file is automatically closed once the last line has been read. It is especially useful for very large files as well, since these can potentially consume large amounts of resources and we would prefer the files close when we have finished processing them.

Reading the lines does little for us on its own; we want to operate on them. The raw strings need to be processed, so we can use the `str` operators `strip()` and `split()` to clean things up a bit. The `strip()` function very helpfully removes extra whitespace and line terminators like `\n` and `\r`, though we can specify other characters we want to remove as well. The `split` function will, by default, split a string into parts (creating a `list` object) by treating whitespace as a

"delimiter", but we can specify our own delimiters as well, such as commas, colons, etc.

```
with open(filename, 'r') as data:
    for line in data.readlines():
        # clean up the line
        line = line.strip()
        # split at commas
        line_parts = line.split(',')
```

We may also want to search the lines for specific patterns, either because we want to skip those patterns or because those patterns indicate useful data in the text:

```
if "pattern" in line:
    # do something
```

Exception Handling

A final note is on the use of the `try-except` block, which is an indispensable tool for handling **exceptions** or errors that arise when Python attempts to execute your code. For example, if you try to divide an `int` type by a `str` type, you will inevitably get a `TypeError` because you cannot divide a number by a text string. If you try to open a file that does not exist, Python will raise a `FileNotFoundError`. How we deal with errors is called **exception handling** and you will find it incredibly important for you coding in this term and the next.

The basic syntax for this type of error handling is

```
try:
    # some statement here
except Exception:
    # error, you must do something else
```

It is always good to anticipate what kind of exceptions you may encounter, and a full list of built-in Python exceptions can be found in the official Exceptions documentation. When Python encounters an error and we have not provided any means for handling it, we will be presented with something called a "traceback". For example, if we try to run the following line of code in a notebook:

```
int(" here is a string  \r\n")
```

we can immediately see that Python is about to complain. Specifically, Python will raise a `ValueError` as seen in Figure 1

Tracebacks can sometimes be difficult to interpret depending on just how many things went wrong, but the general format is still the same: we are given the name of the exception, the line where this exception occurred, and a statement telling us more information

```
int(" here is a string    \r\n")

---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[48], line 1
----> 1 int(" here is a string    \r\n")

ValueError: invalid literal for int() with base 10: ' here is a string    \r\n'
```

Figure 1: A `ValueError` raised by attempting to convert a `str`-type to an `int` type.

about what went wrong. In this case, we see "invalid literal for `int()` with base 10:" and the offending string we tried to convert.

If we are unsure what type of errors we might get, a safe practice is to use the following syntax:

```
try:
    # some statement here
except Exception as e:
    print(e)
```

where we capture the name of the exception in a variable `e` and print it so we know what went wrong. In this situation, as soon as the code encounters an exception it stops executing completely. For this reason, some programmers have taken to using a `pass` statement to skip the offending line code and continue with the rest of the program; as you will see in our workshop sessions, this is *bad Python practice* and can lead to potentially severe problems.

Functions & Modules

You are aware of functions in mathematics, so we will spare the boring details here. Functions in the programming sense are similar, but are perhaps better thought of as procedures or subroutines within a program. In Python we can define functions with the following syntax:

```
def my_function(x):
    y = x*6 + 3
    return y
```

where we pass arguments to the function, the function operates on those arguments, and the function will **return** a value based on our defined operations. We can call this function simply with `my_function(5)` and get a value. This is similar to our normal understanding of functions and the above code is equivalent to $f(x) = 6x + 3$.

We can also define our functions to take multiple arguments. We typically talk about *positional arguments* and *keyword arguments* in this scope. Positional arguments are simply assigned in the order they appear; for example

```
    def my_func(x, y):
    ...
```

would work such that the first value you send to `my_func()` when you call it is `x` and the second value is `y`. For functions of few arguments this is fairly simple, but if we have many arguments we may want to use *keywords* instead.

```
def my_function(x, a=1, b=2, c=3):
    y = a * x**2 + b * x + c
    return y
```

By default this has assigned values to 3 local variables, `a`, `b`, and `c`, so all it strictly requires is the positional argument `x`. These keyword arguments can be overridden, e.g. `my_function(5, a=3, b=2, c=1)`, so that we can exert more control in that way. You have actually already seen this behavior with `open()` in the section on file reading: you provide a filename as the positional argument, and by default `open()` assigns the keyword argument `mode='r'`, which we can choose to override. When using both positional and keyword arguments, the positional arguments must come first.

Using functions helps to ensure our code is **modular** (the programming motto here is "Don't repeat yourself"), and allows us to maintain clarity. Our code becomes easier to maintain and provides a well-defined scope for all of our operations. The modular nature is especially important: I can define a function in one file and call it from another file! In fact, this is pretty much what we do when we import Python packages.

At the very beginning we had you use `conda` to install four Python packages. If we are using an IDE or Jupyter, we can import those packages into our workspace quite simply using any of these four methods:

```
import module_name
from module_name import some_name
from module_name import some_name as name
import module_name as name
```

Any of these approaches is just fine and which one you use depends on what you plan to do. Most people will simply use the shorthand approach, especially for `numpy`, because it is used so extensively that it simply saves time to import it as `np`.

All Python packages (sometimes called modules) are like libraries of modular code that can be used after importing them as above. These modules can contain variables, functions, and classes that ultimately make our lives easier. In our Jupyter notebooks we will be using in the workshop, you will have plenty of opportunity to

explore the functionality of various Python packages.

Plotting & Visualization

How can we visualize data in Python? The most widely-used package for this is the `matplotlib` package, and the easiest way to work with it is:

```
import matplotlib.pyplot as plt

plt.plot(x, y)
plt.show()
```

We can add other arguments to `plt.plot()`, allowing us to adjust the line color, line width, line style, data labels, and so on. We can also fine-tune our control over the figure size and axes using `subplots()`:

```
fig, ax = plt.subplots(figsize=(w, h))
ax.plot(x, y)
plt.show()
```

We can have a plot with multiple axes too, using the `nrows` and `ncols` keyword arguments. To create a 4-panel plot with 2 rows and 2 columns:

```
fig, axes = plt.subplots(nrows=2, ncols=2,
    figsize=(w, h))
ax[0, 0].plot(x, y)
ax[0, 1].plot(...)
plt.show()
```

where the `axes` object is treated like a nested list.

Curve-Fitting

One of the most important tools for us as physicists and engineers is knowing how to fit a model to data. Assuming we have examined our data closely and decided upon what type of model could best describe it, we can tell Python to do the heavy lifting using `numpy` and `scipy`.

In short, let us say we have a function $f(x, \theta)$ where θ represents the parameters of the function. If this is a line, then $\theta = [a, b]$ and this satisfies the function $f(x; a, b) = ax + b$. We want to use $f(x, \theta)$ to model our observable data, and we can do this using a **least-squares** algorithm. This essentially makes a guess as to the parameters θ of the function which best fit the data, compares this fitted function to our data and computes the **residuals**, and continues this process until the difference between the fitted function and the data is minimized.

If we call our observed value y_i, then the residual can be written

$$r_i = y_i - f(x_i, \theta) \tag{1}$$

And the least-squares approach sums up all the squares of these residuals:

$$S(\theta) = \sum_{i=1}^{n} \big(y_i - f(x_i, \theta)\big)^2 \tag{2}$$

The optimized parameters are the parameters which result in the smallest $S(\theta) value$, and we typically do not want to do this by hand. We can, however, define both of these equations as Python functions and use the `scipy.optimize` module to minimize the residuals. The basic syntax is:

```
import scipy.optmize as spo

def my_line(x, a, b):
    return a*x + b

def residuals(parameters, x, y):
    model = my_line(x, *parameters)
    residuals = np.sum((y - model)**2)
    return residuals

best_fit = spo.minimize(residuals,
   initial_parameters, args=(x, y))
```

While this is the basis of countless fitting routines, as it stands this will not give us any information about the errors on our fit, and when dealing with physical measurements we are especially interested in such errors or uncertainties. A better approach is to use the `curve_fit` function, also in the `scipy.optimize` module.

```
from scipy.optimize import curve_fit
par_opt, par_cov = curve_fit(my_line, x, y)
```

This returns the optimized parameters and the **covariance matrix** associated with those parameters, and by computing the square root of the diagonals of this matrix we can extract our uncertainties:

```
uncertainties = np.sqrt(np.diag(par_cov))
```

There are other parameters which can be passed to `curve_fit`, such as boundary conditions and initial guesses, and you are strongly advised to explore the documentation in this regard to gain a better understanding of such usage. Appropriately defined functions are the key to success in this regard, and this might require careful consideration when handling your routines!

Exercises

N.B. These exercises can all be found in the "Python-workshop-exercises" notebook on GitHub. The repo additionally contains the data and image files you will need for Tasks 3 and 4.

1. Print the numbers from 1 to 100 (inclusive), replacing numbers that are divisible by 3 with the string "Fizz" and numbers that are divisible by 5 with "Buzz". If a number is divisible by both 3 and 5, replace that number with "FizzBuzz".

2. Write a code that generates an integer between 1 and 20, and randomly chooses to either square or cube that number. Store this result, and keep track of which result is the smallest and which result is the largest. If the current number is divisible by the previous number, issue a print statement and exit the loop. After the loop has exited, print the smallest and largest of the squared or cubed numbers, state which two numbers caused the loop to break, and inform the user how many loops the program completed.

3. Given an optical spectrum from an astronomical source, write a Python script that parses the data file and produces three plots. Your first plot should simply be the full spectrum. Your second plot should contain full spectrum with a polynomial fit to the baseline of the spectrum overlaid. Your third plot should contain the full spectrum, polynomial fit, and a Gaussian fit to the emission peak. Include appropriate axis labels and units, and ensure all elements of the plots are clearly legible. Your script should print the fitted parameters with their uncertainties to the terminal. As a guideline, structure your script so that it can be called simply with

   ```
   $ python my_script.py filename
   ```

4. Using the ascent data provided for the Apollo 10 launch, write a python script that parses the file and stores the data from the table columns as arrays within a dictionary, and use this dictionary to make plots of the data. You should plot all 11 parameters as functions of time, and make a plot of the Apollo 10 rocket's ground track. More information about this task is provided in the "Exercises" notebook.

Addendum: Classes

A final important (but maybe not essential for us) is a discussion on the concept of **classes**. Python is an "object-oriented programming" language, meaning that it bundles properties, attributes, and behaviors into containers we call **objects** and we create these objects from **classes**. Classes are the blueprints for creating objects, and objects are the digital representatives of real-world things and phenomena. When we **instantiate** an object, we bring it to life by creating a variable to represent its thing-ness. This is necessary because the class only defines what the object *is*, not the unique object itself.

When we construct a class, we typically use it to contain to contain attributes (data) and methods (ways to manipulate data) that govern the properties and behavior of an object. A relevant example we could use here is a satellite. At its most simplistic, we would have a few **attributes** like name, altitude, power, and data storage; the **methods** we define could be power on/off or maybe collect/transmit data. How could we translate this to Python?

```
class Satellite:
    def __init__(self, name, altitude):
        self.name = name
        self.altitude = altitude
        self.power_on = False
        self.data_stored = 0.  # in MB

    def power(self, state):
        self.power_on = state
        status = "ON" if self.power_on else "OFF"
        print(f"{self.name} is now {status}")
```

You are familiar with functions so `power()` needs no explanation; but `__init__()` is probably new to you. This is called a **constructor method**, and it runs automatically any time an object is instantiated from the class blueprint to essentially fill the object with its thing-ness. Each class function must have a `self` argument, as this is how Python refers to the specific object created by the class. We can create a simple satellite with:

```
hubble = Satellite("Hubble", 515)
jwst = Satellite("James Webb", 1.5e6)
```

We can interact with these:

```
print(hubble.name)
print(hubble.altitude)
print(hubble.power(True))
```

which will print the name and orbital altitude, and toggle the power to "ON". However there is still an attribute that we have defined but neglected: `data_collected`. This is a dynamic attribute just like the power status, so we expect that this should change in some way.

Let us create two new class functions that allow us to specify how much data has been collected and instruct the satellite to transmit data. We will consider first what is actually happening. Before we collect any data we need to ensure the satellite is powered on, and when we collect data we need to increment the `data_collected` attribute by some amount, so we will need to specify that amount. When we transmit, we assume for the sake of simplicity that all the collected data gets transmitted, so it needs to reset the `data_collected` attribute to 0 again.

```
class Satellite:
    # previous functions are unchanged

    def collect_data(self, amount):
        if self.power_on:
            self.data_stored += amount
            print(f"{self.name} has collected {
                amount} MB of data.")
            print(f"It now has {self.data_stored}
                MB of data.")
        else:
            print(f"{self.name} is powered off.
                Cannot collect data.")

    def transmit_data(self):
        if self.power_on and self.data_stored > 0:
            print(f"{self.name} has transmitted {
                self.data_stored} MB of data.")
            self.data_stored = 0.
        elif not self.power_on:
            print(f"{self.name} is powered off.
                Cannot transmit data.")
        else:
            print(f"{self.name} has no data to
                transmit.")
```

We can test these new methods:

```
hubble.collect_data(512)
hubble.collect_data(64)
hubble.transmit_data()
```

In terms of the above discussion, when we call either of these methods, the `self` argument is implicitly the object `hubble`, so we do not need to specify that argument. This is still a rather simplistic example, but the benefit of class functions should be fairly clear. These can be as comprehensive as we want, as well: we could restructure our class to describe global attributes and methods that all satellites have, and then we could build further sub-classes of satellites that **inherit** the global attributes and methods from the **parent** class, and each sub-class (or child class, if you would like) is given their own unique attributes and methods to complement these global ones. This concept is called **inheritance** and it is also incredibly important in Python.

In accessing the attributes and methods of the `Satellite` objects we created, we might notice that the syntax is quite similar to things we have already done, for example `np.array` or `scipy.optimize`. This is not by accident: modules in Python are a toolbox that contain all the blueprints for the tools we want to use, as well as "loose tools" (for example, `numpy.sqrt()`) for quick access. When we create an array with `numpy` we are creating an object from a blueprint, and we can access its methods and attributes with things like `my_array.sum()` or `my_array.shape`.

When designing your own programs later in this course, you may decide that creating classes is the best approach for organizing the workflow; or you may decide this is too much work and feel that you will not benefit from its usage. The great thing about Python is that it ultimately does not care: we could design a rigorous data reduction pipeline without using any of our own classes at all and still be just as useful as a pipeline designed with custom classes and objects. Knowing how to use them, though, will enhance your own coding abilities in the end and allow you to better understand the overall architecture of the language.

Recommended Reading:

- A Whirlwind Tour of Python by Jake VanderPlas
- Think Python (2e), by Allen B. Downey
- The Hitchhiker's Guide to Python
- Scientific Python Lectures, by dozens of authors and contributors
- Python Classes, in the Python documentation

GitHub

Another (indispensable) tool we will be using is called *Git*[5], which

[5] https://git-scm.com/

is a "distributed version control system" that allows you or multiple others to simultaneously work on a similar project while keeping track of different versions of the project. It is far more efficient than simply saving multiple versions of a similar file, and it is vastly superior to standard word processing software which can only keep track of a limited number of recent changes (that is, you can only `ctrl+z` or `ctrl+y` so many times in a single document). With Git, you will always have a record of what you've done so you can revert to an older version if need be, and all the different changes you or other people have made can ultimately be merged into a single source, making this particularly useful for programming teams.

There are many platforms on which to use the Git architecture, most notably GitHub and GitLab, and on both of these online platforms you can find just about any open-source project or software you can imagine. For this course we will be using GitHub, so you need to navigate to their site and set up your own account using your school email. Details on the installation and setup of `git` on your local machine will be discussed below.

Understanding the GitHub workflow

There are three major concepts with which you will want to familiarize yourself when utilizing the git architecture: repositories; cloning; and committing and pushing. We provide a brief summary of each below, and will discuss them further in turn.

Repositories (or *repos* in shorthand) are simply a place to put your work. A repo functions essentially like a folder or directory on your computer, which we call the local machine. On GitHub, these repositories are stored in a cloud which we call a remote. The repository for a project contains all of that project's files and revision history – everything you've done is kept here.

Cloning, as the name implies, allows you to make a local copy of a repository (that is, you can save it to your computer). When you clone a repository GitHub pulls down *all* of the data from that project to your machine, giving you access to every version of every file and folder of the project. When you alter the local copy on your computer, you can use Git to sync your local copy to the remote copy. This makes it easy to edit code or fix issues, and also to add or remove entire files or folders that you may not need for your project.

Committing and pushing are the collective processes by which you take all your edits and changes for a project and add them to the remote repository. When tell Git that you want to *commit* certain files, it keeps track of them in your repository and makes a "checkpoint". It is best practice to include useful *commit messages* so that other con-

tributors can have an idea of what changes you made. Once you've committed all your files and folders that you want to be part of your project, you use the *push* command to tell Git to add those changes to the remote repository.

In each repository we begin with a default "branch" (or version of that repository) which we call `main`. By longstanding tradition, programmers are expected to reserve the `main` branch for the final, publishable version of their project or code. If we want to make a change or add something to our project but we do not want to change the primary code, we create a different branch with a descriptive name and make any edits on the code there. If we are working as part of a group, we can create something called a "pull request" that lets the other collaborators know we have proposed a change to the code, and (if all goes well) merge our branch with `main`, as seen in Figure 2.

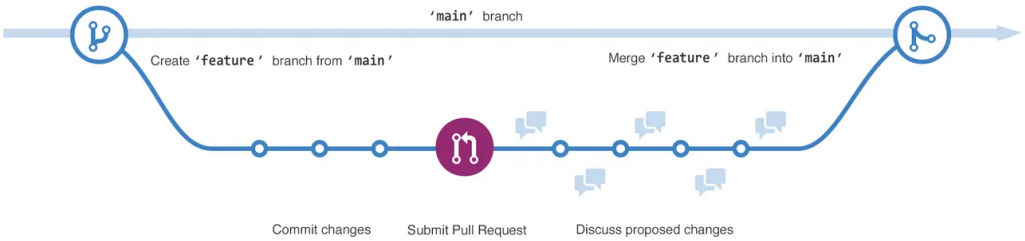


Figure 2: A schematic demonstrating the "branching" of a repository; from GitHub.com.

The branching process can be as simple or as complicated as we like. For example, we could create a branch of `main` called `development` where we do the majority of our work. We merge our `development` branch occasionally with `main` to create different official versions of our program, e.g. V2.0 or V2.1. Something in `development` breaks, so we create a branch called `hotfix` to try to fix the issue. However, during the process we break something else, so we create another branch called `hotfix-hotfix` to address that issue. These two hotfixes get merged into our `development` branch. While working on that, we also decide we want to add a new feature to our code so we create a branch called `feature` to handle this. A rudimentary flowchart of such a workflow is shown in Figure 3.

With larger projects this can become even more complex, where we might have different `feature`-type branches for different software editions we make, and multiple people might have their own `hotfix` branches or even separate branches representing their own team's work. No matter the scenario, the process is the same: we pull the latest version of the project and its branches, we move to some

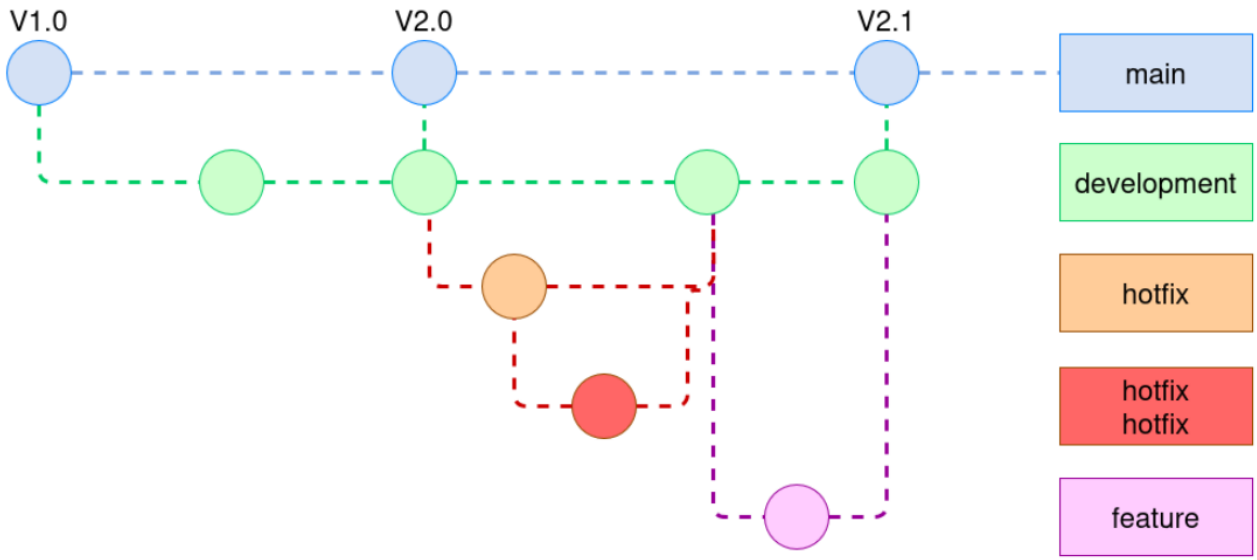


Figure 3: A fairly typical workflow for a software project.

branch we want to work on, we commit and push our work to that branch, and eventually merge all the working code into the primary branch.

Installation & Configuration

To use Git we need to have it installed on our local machine, and it needs to be configured. For this module and other modules in this course, you will maintain your code on an organized repository and submit the repository link to us. As mentioned above, you need to sign up for a GitHub account using your school email (not a personal email). Assignments will be provided to you through BrightSpace as "GitHub Classroom" instances, and when you accept an assignment you will be added to the Classroom and we will have an easy way to see your progress.

The preferred method of installation for this course is through the `conda` environment, though you may choose to use the GitHub Desktop Graphical User Interface (GUI), which is available for Mac and Windows.[6] Once it is installed you will need to configure the Git software with your name and email before you can make any commits to your repo. This is done through the command line:

[6] If you are using the command line on Mac or Linux, then you know what you're doing so you don't need any help here.

```
$ git config --global user.name "Firstname Lastname"
$ git config --global user.email "YourStudentEmail"
```

Note that using your email here can pose a security risk. To mitigate this risk, you are encouraged to use the GitHub `noreply` email address that GitHub provides to users. Go to `github.com` and navigate into your account settings, where you can select "Keep my email address private" in the Email settings options. More about this can be read on the GitHub docs page "Setting your commit email address".

Next you need to configure a **personal access token**. You can follow the steps on the GitHub documentation to do this. If you do not wish to re-enter your credentials every time you make a push or a commit, you can tell Git to store your credentials on your local machine:

```
$ git config --global credential.helper store
```

Basic Operations

Above we discussed some concepts that are fundamental to the git workflow. When working with Git, there are some essential command-line operations that you should devote to memory.

- `git add` – add a new file to the repository
- `git rm` – remove a file from the repository
- `git mv` – rename a file in the repository
- `git fetch` – check the remote repository for changes
- `git pull` – pull the most recent changes from the remote repository into your local repository
- `git push` – push local changes to the remote repository
- `git commit` – save your changes as a new version in the repository

This is not an exhaustive list, and you are encouraged (expected, even) to consult the additional materials on the module BrightSpace page.

So, what do we do with all of these, and how to we interact with GitHub? If you followed the tutorial documentation on the GitHub site (which you should definitely do) then you will have noticed that on every GitHub repo webpage there is an option to clone the code, as in Figure 4. The two most common options are to clone a repo over HTTPS or over the *secure shell protocol* (ssh), but you can also use the GitHub CLI, GitHub Desktop, or even download a ZIP archive

if you want. For one-off software downloads the ZIP archive is okay, but if you are consistently pulling from and pushing to a repo you should really use the `git clone` method.

With HTTPS, you will be prompted for a username and password unless you have stored your credentials. When using the UCD Wireless network, we have found that sometimes HTTPS is more reliable due to the UCD firewall. If using ssh, you will need to set up a "public key" that your local machine will use to authenticate all of your transactions.[7]

Now, you simply open a terminal and type in your commands. If we want to clone the Introduction to GitHub repo using the HTTPS protocol, then we can type:

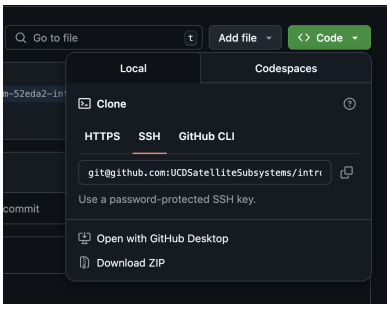


Figure 4: The `clone` options available on the browser interface of a GitHub repo.

[7] see Connecting to GitHub with SSH in the GitHub documentation.

```
$ git clone https://github.com/skills/introduction-to-github
```

All going well, you should have successfully cloned into the `introduction-to-github` repo. This will automatically create a new folder in whatever working directory we are using – for example, if I was in my "My Documents" folder then I will see a new folder where the repo was cloned. Whenever I use my Git commands, I want to be in that newly created folder.

There is a very important thing to point out here, however. This just creates a local repo that points to the original URL I cloned, and if I try to push or commit any changes I might find it difficult because I am not the owner of that GitHub repo nor have I been given any permissions for contributing. So, how can I make changes and commit them to a repo under my own username? To do this I need to *fork* the repository – this creates a copy under my own username – and then clone the fork. For the majority of our classwork you will not have to worry about this: when you accept the GitHub Classroom assignments, it automatically forks the repository under your own username.

Try it yourself: go to the `introduction-to-github` repo, create a fork under your username, clone the fork. In that directory, create a new text file with "Hello, world!" in the body and commit and push it to your repo. Did it work?

Recommended Reading:

- Pro Git, by Chacon & Straub
- Using GitHub, on the GitHub documentation page

Using LLMs for coding

Since the first release of ChatGPT in 2022, Large Language Models (LLMs) have been rapidly adopted across many areas of society. It is clear that LLMs will continue to improve in capabilities over the years to come, and that they are set to feature in all areas of society (including science) in the future. With this in mind, we do not wish to make a blanket ban on AI, but rather invite students to consider their use of this, and to ensure that if they use LLMs or other tools, they do so responsibly.

LLMs have proven extremely good at writing code. In fact, if you ask an LLM to write some Python code to solve a textbook coding problem (for example, writing an implementation of bubble sort), it will likely do so correctly. This performance most likely reflects the structured nature of coding languages, as well as the vast amount of example code on which LLMs have been trained. Where LLMs will perform less well is on problems that are more niche, or where there are multiple steps involved. The danger is that an AI may produce some code that perhaps works, but misses critical steps or incorrectly implements an algorithm that an inexperienced user may miss[8]. In these cases, what is necessary is to break the problem down into small chunks that the LLM *can* handle.

[8] This is an excellent example of the proverbial "chimp with a machine gun"

As an example let us ask ChatGPT to write some code to do point spread function (PSF)-fitting photometry on some HST/ACS images (don't worry if you don't know what this is – it's a fairly standard type of data analysis that one would run on Hubble Space Telescope images). If we prompt ChatGPT as follows

```
I would like some code to do PSF-fitting photometry
    on HST ACS images. Images will need to be
   aligned, a PSF model built and photometry
   calibrated onto a standard system
```

then what we get back is code that will run, but give completely incorrect results. More specifically, the prompt to "align the images" results in code that assumes the World Coordinate System information in each image is correct, while in reality there will be ∼few pixels offsets that must be accounted for. Similarly, the code builds a model of the PSF from *all* sources in the field, whereas best practice is to use bright, isolated stars to model the PSF. Again, the key point is that you get something that is superficially correct and will run, but that has serious and fundamental errors.

To avoid this, if one is using LLMs to write code then it necessary to break the problem down into sections. The overall structure of the solution is generally something that the user can (and should) design.

In this case, before beginning we might decide that our code needs to align pairs of images, build a PSF, detect sources, fit a PSF to each source, and calibrate the resulting value. Here, one could prompt ChatGPT with more granular instructions, and iterate to build code rather than doing this in a single step. For example, just to align the images we could use the following prompts

```
I would like to write some python code using the
   astropy library to read in a fits file.
```

then

```
I would like to calculate the pixel offset in x and
    y from a second fits image by doing a two-d
   cross-correlation with scipy.
```

then

```
Finally, I wish to measuring the position of the
   cross-correlation peak by fitting a 2d Gaussian
   to it.
```

As you can see, we are building our code in a much more iterative fashion. We are giving explicit instructions on how we want ChatGPT to solve the problem, and we are also doing it in a way that we can look at the output at each step to see if it as expected, rather than simply seeing the final result[9].

[9] This has been described as vibe coding.

Even doing this, LLMs can still make mistakes. So, it is critical to build tests and "sanity checks" into your code to help ensure that the output is correct. Taking the example above, one could easily plot the two images, and the shift, and visually confirm that they are aligned. Also, keep in mind that even if the code that is produced by an LLM works, it may not be particularly "good" code, or an optimal or efficient solution.

It is also important to remember that while LLMs have been trained on a vast corpus of code, the more obscure and niche the problem, the less useful they will be. Usually, your specialist expertise will still be needed to write the core of whatever code is needed: designing an algorithm to model the temperature increase when a thruster fires, or inferring rotation rate from solar panel data on a tumbling satellite. Remember that this is where your value and skillset lies, and that while it will often be fine to use LLMs for much of the boilerplate code surrounding a problem (for example, reading and parsing some datafiles into a `numpy` array), the core of what you are trying to do will usually require human expertise and experience.

The guidance above pertains to using LLMs to write code. The advice for writing text is quite different. It is important to keep in mind that LLMs work by forming associations between tokens and

patterns. As they do not have any understanding of what they are producing, it is inherently risky to use them to write introductions, or indeed any other parts of a report or document. While code produced by an LLM can be tested, and we can satisfy that its outputs are correct, we cannot do such a test for, say, a description of how a Hall thruster works.

Aside from the major problem with using LLMs for writing, namely that it has no *understanding* of what it outputs[10], there are other more subtle downsides:

[10] See this article for an entertaining discussion on this point.

- Over the course of your MSc, we want you to acquire a broad range of skills relevant to the space sector. One of the best ways of learning information and reinforcing it is by explaining to others, whether in the form of an oral presentation or writing a report or essay. Using LLMs to summarize a topic negates this - even if you feel you know what you are writing about, you are more likely to retain this information into the future by writing it down yourself.

- It is also easy to develop a habit of turning automatically to LLMs, even when they are not the best tool to use in that moment - for example, if there is a human expert readily available.

- Some LLMs will often structure output in condensed snippets or bullet points that, when used in academic or scientific writings, can severely undercut the importance of the point the author is trying to convey.

- Most industry and government bodies have very strict rules around data confidentiality and security. Using an LLM from a third party can result in loss of proprietary information.

As a final note, we may also be concerned about the the environmental impact that systems support LLMs can have. For example, a single Google search uses about 10% of the Wh required to ask ChatGPT the same thing.[11] While this may amount to comparatively very little for a single user, as the number of users of LLMs grows the energy usage increases significantly; according to the CSO[12], as of 2024, data centers in Ireland accounted for 22% of all electricity usage in the country – an increase of over 500% since 2015. While of course the overall impact of LLMs and data centers is far beyond the scope of this course, as scientists we should nevertheless seek to understand the tools we use so that we can make informed decisions about our relationship with those tools.[13]

[11] See here for reference, as well as the MIT Technology Review series on the subject.

[12] CSO Statistical Release, ISSN 2811-5422

[13] See, if you are interested, The Cloud is Material for one opinion (of many) on this subject.

Key points for using LLMs in this module

- An LLM is a tool. Always use it consciously, deliberately, and with a clear understanding of what you are doing.
- Any use an LLM for assistance with coding must be clearly identified and acknowledged.
- Do not use LLMs for writing or generating text in reports or assignments. If you use an LLM for language editing, then you should identify this, and also submit the (non-edited) version of the report alongside any language-edited text.

Note that each module will have its own rules regarding the use of AI, and you should follow these.

Optical and other detectors

Introduction

In the days of yore, astronomers relied solely on their eyes and artistic abilities to record their observations of the sky. In antiquity it would have literally been their naked eyes doing these observations; with the invention of the telescope, details were able to be refined, but there was still no way to *record* these measurements apart from manual sketches. The limitations of drawings do not need to be stated here. A more important limiting factor is the inefficiency of the human eye at processing light: we have evolved useful photoreceptors to assist us in mediating our visual sensations, but they are only sensitive to a narrow range of the electromagnetic (EM) spectrum (about 400-700 nm; see Figure 5), and evolution has provided us with a peak response to green light, around 555 nm.[14]

By the 19th century, the use of light-sensitive materials to permanently fix an observed image to an external medium led to the invention of photography. Initially relying on solid plates (like the daguerreotype in 1839, which captured the first photograph of the moon), early experimenters developed various techniques that exploited both optics and chemistry, and various technological advancements eventually resulted in the creation of photographic film by the end of that century.[15]

[14] Additionally, conditions like color blindness and age can alter our experience of visual stimuli, further complicating the objective recording of *structure* observed in celestial objects. See Kawamura & Tachibanaki (2012) for a review on the subject.

[15] See the Photographic Technologies page at the Royal Collection Trust if you are interested in these early techniques; see also The History of Astrophotography on the High Point Scientific Astronomy Hub.

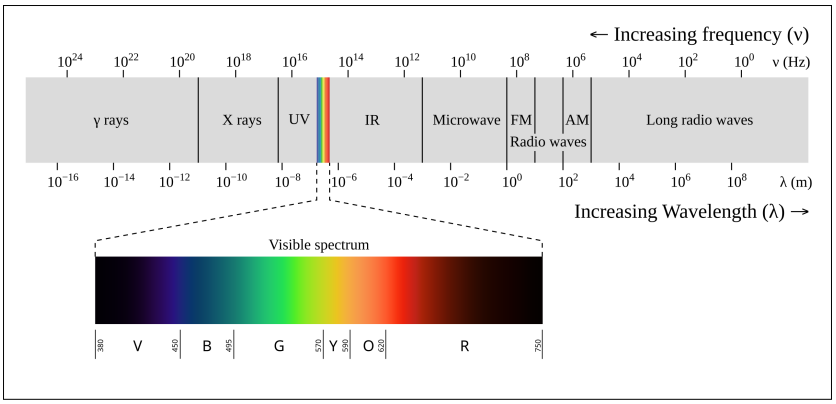


Figure 5: A visualization of the electromagnetic spectrum, highlighting the region of visible light to which the human eye is sensitive. Illustration by Philip Ronan, CC BY-SA 3.0, obtained through Wikimedia Commons.

Unfortunately, photographic plates and film are not much more efficient at processing light than the human eye, although they do receive light over a broader wavelength range. Standard film is sensitive to wavelengths around 300-1100 nm, while clever chemical techniques could be used with photographic plates to see x-rays and γ-rays! For astronomical purposes, photographic plates offered the great benefit that they could provide a wide field-of-view (FOV) and high resolution, which was useful for standardizing observations across telescopes and producing large sky surveys. Another advantage to these techniques over the human eye is that we could control the exposure time, meaning more photons could be collected to get a better image; by contrast, the exposure time of the human eye is fixed by biochemical processes regardless of how long we look at something. Plates and film do have the unfortunate drawback of non-linear behavior, however: if we think of the medium like a sponge that soaks up photons, the more photons it absorbs the more difficult it becomes to continue absorbing photons. In this way, bright light does not necessarily create a proportional increase in the exposure, which can make it difficult for astronomers to perform a quantitative analysis of their observations.

Everything changed in the 1950s and 1960s when researchers at Bell Laboratories developed metal-oxide-semiconductor (MOS) technology, which laid the foundation for two groundbreaking devices in image sensing: the so-called "charge-coupled device" or CCD; and the complementary MOS or CMOS. CCDs proved incredibly efficient at capturing light over a broad wavelength range, which led to the wide-spread adoption of the technically across a variety of fields from biology to astronomy. Early on CMOS image sensors had quite poor performance compared to CCDs in terms of image quality (though they excelled in power efficiency), but in recent years major developments have led to CMOS technology (and others) actually surpassing CCDs in many applications. In the following discussion, we will explore the basic functionality and characteristics of CCDs and CMOS detectors, and explore their usage in the context of astronomy and space science.

Detector Fundamentals

A detector, in our context, is any device that we place in an instrument or telescope that allows us to measure incident photons. These photons can be at any wavelength, and we choose the type of detector based on the energy range we wish to observe. We are generally interested in a few other characteristics as well, namely the **spectral response** of the detector and the **quantum efficiency** of the detector.

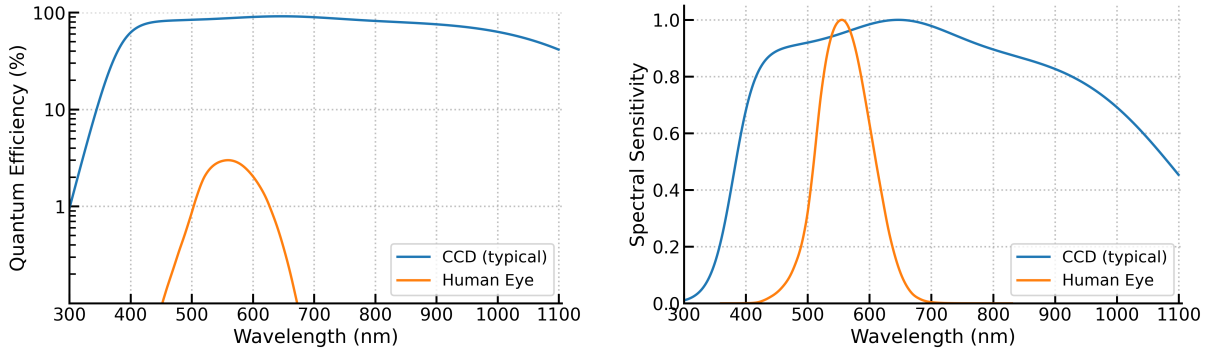
The spectral response is like a sensitivity in that it tells us how effectively the detector is able to distinguish different colors (that is, light at different wavelengths), while the quantum efficiency tells us how good the detector is at converting absorbed photons into an electrical signal. We can express the responsivity of a detector by

$$R_\lambda = \eta \frac{e}{hf} \tag{3}$$

where e is the charge, h is Planck's constant, and f is the frequency of the optical signal. The parameter η is the quantum efficiency, defined simply by

$$\eta = QE(\%) = \frac{\#\text{ of electrons produced}}{\#\text{ of photons} - \text{absorbed}} \tag{4}$$

With semiconductor materials we often see silicon, though other elements can be used as well. Optical detectors feature a grid of photo-receptors called pixels that absorb light and produce a corresponding electrical current that is directly proportional to the energy of the absorbed light. CCDs and CMOS detectors are incredibly efficient – better than 80% in the peak response range – and are responsive across the entire spectrum. A comparison of the responsivity and efficiency of a typical CCD and the average human eye can be seen in Figure 6.



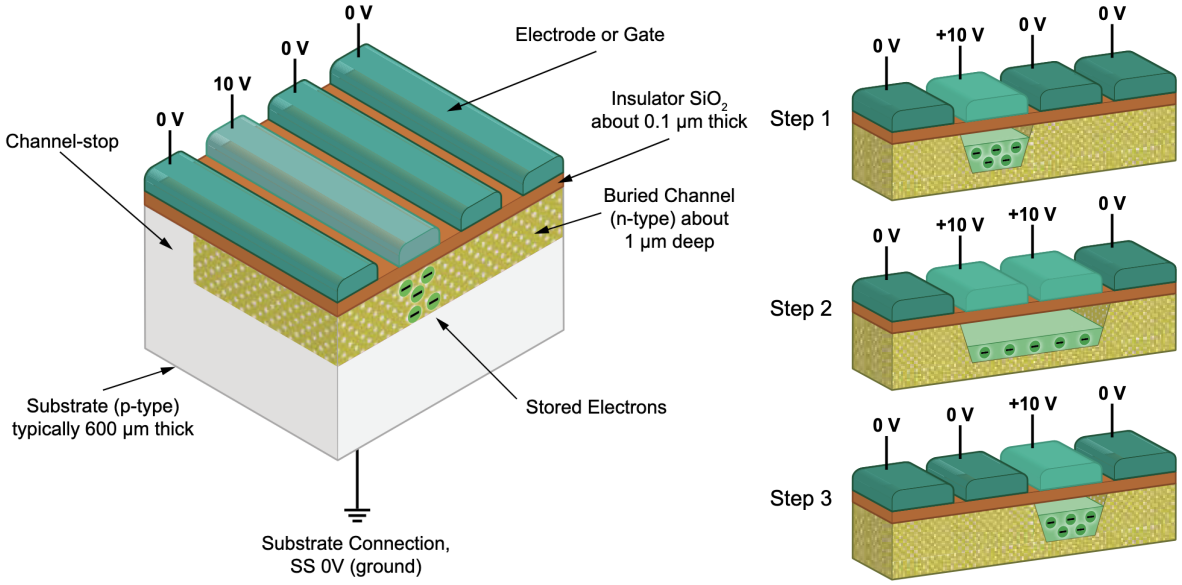
(a) (b)

Figure 6: Graphs demonstrating (a) the typical quantum efficiencies of a CCD detector and typical human eye, and (b) the spectral response of a CCD detector and typical human eye, both as functions of wavelength. Data for the eye adapted from the CIE spectral luminous efficiency for photopic vision table (2019).

How do CCDs and CMOS achieve such high efficiency? At the most fundamental level, they exploit the photo-electric effect, where a material emits electrons after absorbing photons that exceed a threshold energy that we call the *work function* of that material. This means we can keep track of both the energy and the amount of incident photons on a detector. By fabricating these detectors from semiconductor materials, we can tailor the band gap of the detector so

that it is conducive to carrying electrical current so long as the incident light is above a threshold energy. (You are encouraged to [re]familiarize yourself with valence bands, conduction bands, and band gaps.) Silicon for example has a band gap of 1.26 eV, which allows us to minimize the effects of thermal excitation and know the wavelength ranges to which the detector is sensitive ($E < 1.26$ eV corresponds to $\lambda < 1100$ nm).

The surface of a CCD chip is typically a grid of small photo-receptor pixels, which are just arrays of capacitors or photo-diodes each connected to electrodes. A pixel will convert light into an electronic charge and store the charge in a potential well to prevent it from spilling over into neighboring pixels. By adjusting the voltages on the electrodes (called **clocking**), the charge can be transferred from pixel to pixel (**charge coupling**) until reaching a serial register at the end of the row. This charge is converted to an analog voltage, and an analog-to-digital converter (ADC) uses the voltage to digitize the number of counts in each pixel and stores it to a disk. A cartoon of a small CCD section can be seen in Figure 7.



(a) (b)

Figure 7: Panel (a): Schematic of a section of a CCD pixel with four electrodes. The channel stop prevents charge leakage from one pixel column to another. Charges are stored in a buried channel separated from the surface by an insulator. Panel (b): Charges are moved across a pixel through clocking. Images courtesy of Teledyne Imaging.

You can imagine that a CMOS detector works in much the same way: light strikes a pixel, the photon energy is converted to a proportionate number of electrons, and some conversion occurs that allows us to read the corresponding value. However, with the CCD the charge is moved accross pixels before being read out, whereas a CMOS detector allows each pixel to locally convert the charge to a digital voltage which is read out directly through a series of

electronic switches. This provides a very rapid and power-efficient operation that also allows pixels to be read individually; while this historically came at the cost of high noise and low image resolution, recent advancements in the technology have seen it surpass the CCD in performance and quality.

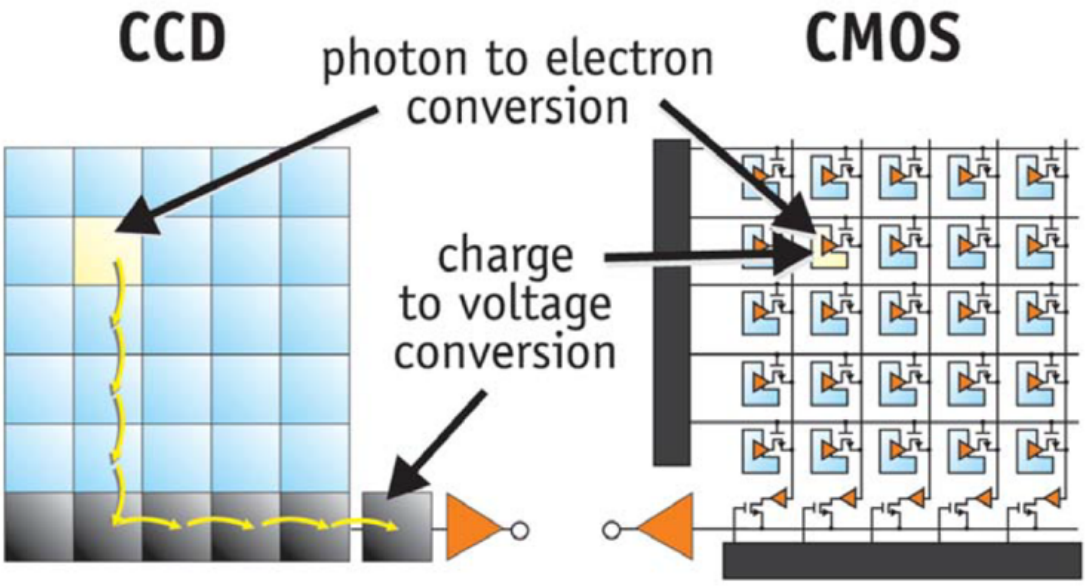


Figure 8: A cartoon demonstrating the charge-to-voltage conversion process for CCDs and CMOS detectors. Image courtesy of Stefan Meroli/CERN.

Filters

We have seen that CCDs and CMOS detectors have a broad wavelength range and significant responsivity. When exploring the physical and morphological *structure* of a terrestrial or celestial object, we generally want to narrow our vision to particular colors. For example, if we measure the brightness of a star in blue light and compare it against its brightness in green light, we can actually use this information to determine where the star fits on the stellar evolutionary chart we call the Hertzsprung-Russel Diagram. We could also look Earth-ward and map vegetation and water vapor using the same principles. We accomplish this using **bandpass filters**, which only allow light from a certain range of energy to pass through. Observatories like the Hubble Space Telescope (HST), the James Webb Space Telescope (HST), and Sentinel-2 allow astronomers and space scientists to use a number of filters on their imaging instruments, which are characterized in terms of their **throughput** or the percentage of incident photons per wavelength they allow to pass through. An example of four well-known HST filters are shown in Figure 9, and a plot of Sentinel-2's spatial resolution as a function of wavelength is

shown in Figure 10.

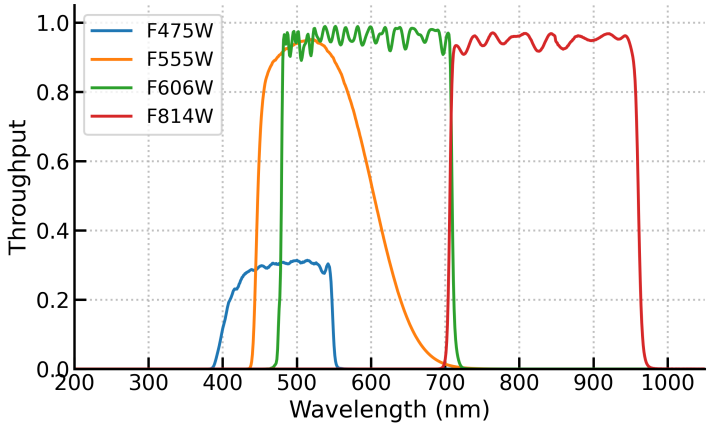


Figure 9: Transmission throughput curves for four HST filters, which are named based on the central wavelength and bandpass width (e.g. **W**ide or **N**arrow. For example, F475W signifies it is a filter with a central wavelength of 475 nm and wide bandpass.

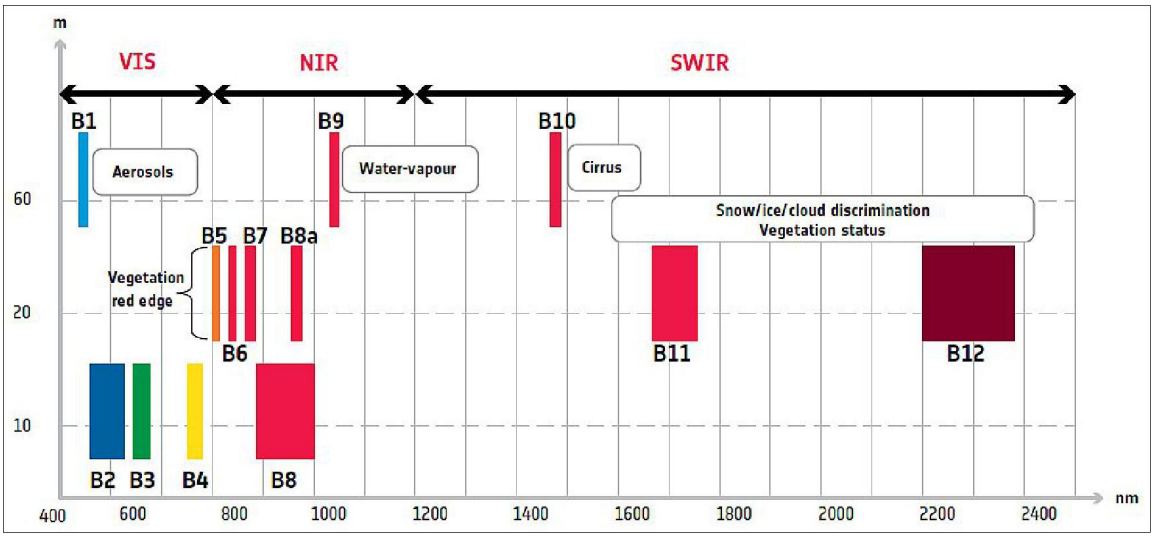


Figure 10: Spatial resolution versus wavelength for the Multi-Spectral Imager (MSI) on-board Sentinel-2. The 13 spectral bands span the visible through the IR. Image courtesy of the eoPortal.

Data Reduction

Simply taking a picture with a detector is not sufficient to acquire science-ready data. To use a raw image we first need to **calibrate** the exposure, and this process is done at three levels:

- **Detector level**: CCDs are not perfect and all exposures contain noise that needs to be corrected. We use calibration images or frames such as *bias frames*, *dark frames*, and *flat field frames* to compensate for this noise.
- **Instrument level**: telescopes are not perfect either, so we need to correct for artifacts and noise effects introduced by e.g. the optical setup or filters, such as distortion, vignetting, and cosmic rays.
- **Physical level**: the digitized values produced by a detector need to be converted to meaningful physical units. For space observations, this typically involves *astrometric calibration* and *photometric calibration*; for a telescope like Sentinel-2, this requires *radiometric calibration* and *geometric calibration*.

The raw image itself is actually a combination of several factors:

$$\text{raw image} = \text{bias} + \text{noise} + \text{dark} + \text{flat} \times (\text{sky} + \text{stars}) \tag{5}$$

$$\text{stars} + \text{noise} = \frac{\text{raw image} - \text{bias} - \text{dark}}{\text{flat}} - \text{sky} \tag{6}$$

Nearly every instrument will bundle calibration files with science observations, and these calibration files (typically numbering in the hundreds) are produced quite regularly. Because all images have noise that we cannot completely remove, we want to combine as many images as we can as this will reduce the noise in the final "stacked" image by $\sim N^{-1/2}$ with N being the number of frames we combined to make our stacked image. This stacking is simply a median combination of all the images of that type.[16]

[16] We prefer the median over the average as this mitigates outlier values.

We mentioned quite a few types of images and calibrations above. In short:

- **Bias frames** compensate for electronics readout noise; these are short exposures with a closed shutter (that is, no incident light).
- **Dark frames** compensate for thermal noise; these have the same exposure time as your science observations but with a closed shutter.
- **Flat field frames** compensate for variations in pixel sensitivity across the detector by applying a constant, uniform illumination. This also helps correct the instrument-level issues mentioned above

We also have a few different types of calibrations depending on whether we are handling images of space or images of the ground. Whichever type of observation we use, we are primarily interested in two things: calibrating the coordinates, which maps the pixels to some absolute geometric location; and calibrating the brightness, which relates the digital counts to an absolute reference frame such as flux, radiation, or reflectance.

For space observations, we use **astrometric** calibration to map the physical pixels to coordinates in the plane of the sky, called "right ascension" (RA, horizontal component) and "declination" (DEC, vertical component). This is typically done using sky catalogs. **Photometric** calibration uses standard stars with known magnitudes (brightness) to calculate the *zero point*[17] of a detector and convert the detector counts to a physical brightness we call the *standard magnitude*.

[17] That is, the magnitude of an object that produces one count per second.

For Earth observations, we use **geometric** calibration based on highly detailed world maps to assign physical coordinates to pixels. This is done alongside **radiometric** calibration, which is somewhat analogous to photometric calibration above. With radiometric calibration, the effects of the atmosphere are much more important, so it is critical to have maps of aerosol content, cloud cover, and water vapor, and correlate these with solar irradiance models and radiative transfer models in order to convert raw values to physically meaningful surface reflectance.

Bad Pixels

A final note here about the data reduction process is handling bad pixels and, for space observations, cosmic rays. As the name implies, bad pixels are literally just pixels that do not work properly. This can mean that they either do not measure any light because they have broken, or they suffer too greatly from the effects of dark current such that dark calibration cannot compensate properly. We can try to identify hot pixels by comparing two dark frames of different exposure times and finding where the count rates are too high. For bad pixels we can use the flat frames, again with different exposure times, and compare them as we did the dark frames, though this process is a bit more complicated. Either way, the result is that we make a *mask* to hide the hot or bad pixels so that we can always keep track of them throughout our calibration and analysis, or in some cases interpolate the bad pixels based on neighboring values.

For space observations, we must also contend with cosmic rays, which are high-energy charged particles that insinuate themselves into nearly all astronomical images. The good news is that these

events are random, so when we take 50 or a 100 calibration images, even though all of them will likely contain cosmic rays, these cosmic rays will not be in the same pixels in every image. Additionally, they will not have the same shape as stars because they result from the local deposition of energy in a pixel, rather than the passage of photons through the telescope optics, and so do not follow a Gauss-like distribution of signal across pixels. For calibration images, we can typically rely on a median combination of multiple exposures to mitigate or even completely remove (if we are lucky) cosmic rays from the final image.[18] We could also use sigma-clipping or median-clipping, which both aim to detect and suppress outliers from the data.

[18] Assuming, of course, that there are enough exposures and these exposures all have the same units.

For science images it is a bit more complicated, so much so that multiple robust algorithms have been developed (such as L.A.Cosmic; see van Dokkum, 2001) to address it. We will not worry so much about that here.

Instrument Response

Sources emit photons at random, and the detection probability is governed by Poisson statistics; however because we are dealing with large numbers this Poisson distribution basically becomes approximately Gaussian:

$$P(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(k-N)^2/2\sigma^2} \tag{7}$$

This distribution plays a large role in determining the "shape" of an object we observe as translated by the detector and the telescope. Both the optical system (mirrors) and the electrical system (CCD) are limited in how well they can resolve or "pick out" the detail of an object and distinguish it from other nearby objects. When we aim the telescope at a small, localized source (like a very distant star) the beam of light is"spread out" in the image. We describe how this light is spread out in terms of the Point Spread Function (PSF) of a telescope, which is essentially the impulse response function of the system that gives us an apparent surface brightness profile.

In reality the mathematical formulation of the PSF can be quite complicated, and is affected by mirror aberrations, beam alignment, charge diffusion in the CCD, and (for terrestrial observations) atmospheric conditions. For instruments like the Hubble, JWST, and SENTINEL, much work has been done to derive empirical expressions for the PSFs of those instruments, and data reduction pipelines typically include tools which attempt to correct the images using the PSF. For our purposes we're interested in the shape, which we can approximate as a two-dimensional Gaussian:

$$G(x,y) = A \times \exp\left(-\left(\frac{(x-x_0)^2}{2\sigma_x^2} + \frac{(y-y_0)^2}{2\sigma_y^2}\right)\right) \tag{8}$$

however, in the context of astronomical imaging this is still a rather gross simplification. A more accurate distribution for modeling the PSF is the Moffat function:

$$M(x,y) = A \times \left(1 + \frac{(x-x_0)^2 + (y-y_0)^2}{\gamma^2}\right)^{-\alpha} \tag{9}$$

where γ functions similar to the Gaussian σ (that is, the core width of the model) and α is the power index. The FWHM of the Moffat function is:

$$\text{FWHM} = 2\gamma\sqrt{2^{-\alpha} - 1} \tag{10}$$

The benefit of the Moffat function for space applications is that it better describes the shape of the "wings" of a source, which is to say how the light distribution into surrounding pixels behaves. It is also of great use for ground-based observations as the γ and α parameters are sensitive to atmospheric conditions.

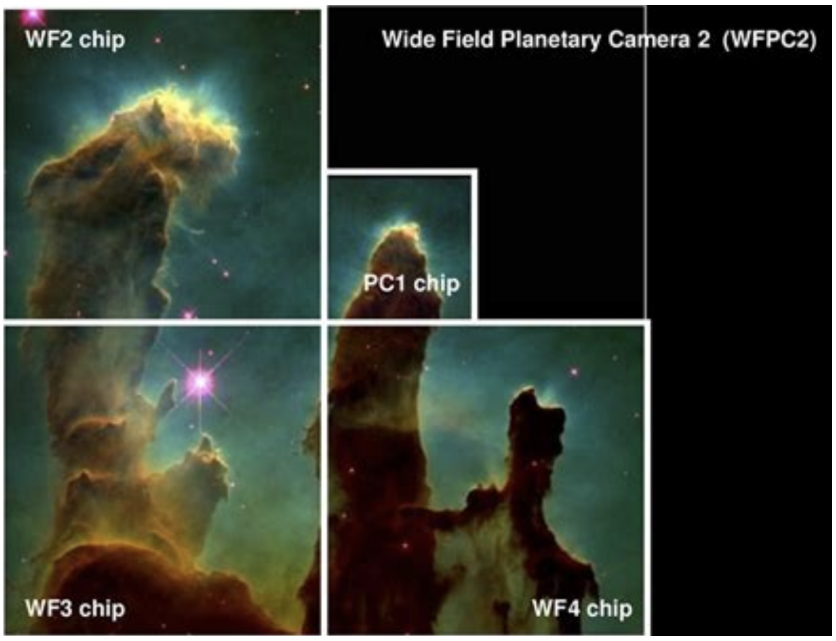
HST images

The *Hubble Space Telescope* (HST) is probably familiar to you already. For those who are unaware, it was developed by the National Aeronautics and Space Administration (NASA) and European Space Agency (ESA) as a low Earth orbit space observatory, and was launched on 24 April, 1990. For 35 years it has been at the forefront of astronomy, and has provided such a wealth of data that agencies like ESA and NASA regularly employ astronomers just to look at old archival data! At the moment, the four primary instruments still in service are:

1. The Advanced Camera for Surveys (ACS);
2. the Cosmic Origins Spectrograph (COS);
3. the Space Telescope Imaging Spectrograph (STIS); and
4. The Wide Field Camera 3 (WFC3)

The predecessor to the WFC3 was the Wide Field and Planetary Camera 2 (WFPC2), which was in operation between 1993 and 2009 before being decommissioned. This instrument consisted of four 800×800 pixel CCDs, three of which are arranged in an "L"-shaped pattern (these are the Wide Field Camera portion) with a fourth CCD

for the Planetary Camera portion. The Planetary Camera chip has a better resolution than the WF chips[19], but when these are all combined into a single image, the pixels are all resampled to the same spatial scale. The result of this is that the PC chip appears "sharper" in its quality than the WF chips. An example of WFPC2 data is seen below in Figure 11.



[19] It is actually more correct to say that the PC chip has better sampling than the WF chips. The pixels in the PC chip are 50 milliarcsec rather than 100 mas on the WC chip. This means that the diffraction limited resolution of the HST optics can be fully sampled by the PC chip, while the WF detectors will undersample it.

Figure 11: An image of the famous Pillars of Creation as seen by the WFPC2. The staircase pattern is seen clearly, and the regions of the different chips are overlaid on the image.

FITS files

Like nearly all digital astronomical data, the data produced by these instruments are stored in standardized files which follow the Flexible Image Transport System (FITS) protocol. FITS files store their data in a multi-dimensional plain-text form as "header data units" or HDUs and combine them into an HDU list. Each HDU is stored as an extension and would contain the physical information (counts, fluxes, etc.; the numerical data) as well as a header that contains all the metadata about the physical information. A visualization of this can be seen in Figure 12.

Typically we have a `PRIMARY` HDU which is a metadata header that contains all the information about the telescope, the observation ID, observation conditions, and other parameters that describe the overall data. The first extension is typically a `SCI` HDU, which would have the image itself as a `DATA` card and more metadata as a `HEADER` card. There are also other extensions that contain, for example, un-

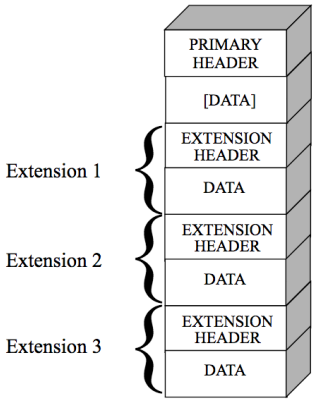


Figure 12: The building blocks of a FITS file. Image courtesy of the Space Telescope Science Institute.

certainties, data quality, and so on, but for 2D images these two are the most common. In the workshop notebooks you will learn how to access these files and their data using Python.

Photometry

When dealing with astronomical sources, we are particularly interested in the brightness of the object. The amount of light that falls into the detector while using a particular filter tells us about the flux of the source,[20] and we can use this in conjunction with the shape of the PSF (and some other instrumental considerations) to glean important information about the nature of the source.

[20] Technically, we are dealing with the *spectral flux density* but many lazy astronomers (one of these authors included) just call it the flux.

A common practice is called **photometry**, which is a way in which we measure the brightness of an object so that we can derive other interesting parameters. At its most basic, photometry is simply "adding up" all the photons within an aperture or circle of some radius focused on a source. To get an accurate estimation, the aperture should be large enough to capture the entire source and its surrounding area, as this allows us to estimate a local background, and additionally provides some local characterization of the noise around that source. At the same time, it should not be so large that signal from other sources contaminate our measurement, so caution should be exercised. A common practice is to use a so-called "annular aperture", in which one circular aperture is focused on a source and small ring surrounds it without overlapping to estimate a background. An example of this is seen in Figure 13.

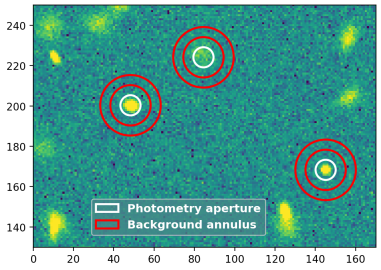


Figure 13: An example of an annular aperture. Small circular apertures (shown in white) are focused on the source, while smaller annuli (shown in red) encompass the local background. Image from the photutils documentation.

When we have counted up the signal counts and corrected for background and noise, we can convert our signal sum into an instrumental magnitude. This is easily found by

$$m = -2.5 \times \log\left(\frac{\mathrm{DN}}{\mathrm{EXPTIME}}\right) + \mathrm{ZP} \tag{11}$$

where `DN` is the data number (typically counts or electrons), `EXPTIME` is the exposure time, and `ZP` is the zero-point of the instrument. This zero-point provides us with a path to translate from counts to a photometric system to a physical measurement like flux density. We can use either the so-called ST magnitude system, where flux density is expressed per unit wavelength, or the AB magnitude system, where it is expressed per unit frequency. For optical observations, wavelength is easiest, and we can calculate our ST magnitude by

$$m_{\mathrm{ST}} = -2.5 \log F_\lambda - 21.10 \tag{12}$$

where the flux F_λ is in units of erg cm^{-2} s^{-1} Å^{-1}. For HST observations, the headers will typically contain information about the

zero-point and conversion factor to transform data units to a flux density. We will work with this more in our workshops.

These steps only provide us with the *apparent* magnitude. To get an *absolute* magnitude (or intrinsic brightness), we need to know the distance to the star. When that is known, we can calculate the absolute magnitude with:

$$M = m + 5 - 5 \log_{10} d \tag{13}$$

where M and m are the absolute and apparent magnitudes, and d is the distance in parsecs.

Why do we care about magnitudes? Most simply because our magnitudes are a direct window into the physical conditions of the star. In fact, using magnitudes as determined from various filters is how we construct the HR Diagrams we briefly mentioned above in our section on filters. You can read all about them on the CHANDRA site. If we recall the blackbody curve from undergraduate physics, we will remember that hot stars emit light most strongly at short wavelengths (blue and ultraviolet; B-band for short) while cooler stars emit most strongly in the visible and infrared (the V-band).

When we use two or more filters to get the magnitude of a star, we will notice that the magnitudes in each filter are slightly different; this so-called "color excess" (the difference in magnitudes between filters) tells us whether the star is hot or cool. As a general guideline a $B - V$ color excess < 0.5 is a hot star, while a $B - V$ excess > 2.0 is a cool star. This incredibly simple parameter tells us about the relationship between the brightness and temperature of stars and how this relates to their evolutionary cycle.

There are many other reasons why we might care about flux densities and brightness in specific filters, but for the moment let us content ourselves with the evolutionary track as observed through an HR Diagram.

Assignment

Exercises

You will be working with observations of the loose globular cluster NGC 1261 as observed with the WFPC2. This object was surveyed as part of a large legacy survey project aimed at better understanding stellar populations, which is detailed in Piotto et al (2015). These data are stored in the FITS image format, which you can load in Python using the `astropy` module. You are tasked with writing a Python script to accomplish the following:

1. Calibrate the three raw images provided by your instructor using the associated calibration files;
2. Stack the calibrated images to remove cosmic rays;
3. Compare your calibrated files against the science images in the F336W filter

While it is not part of the submittable assessment, you should also stack the science images to remove cosmic rays before comparing them to your own calibrated files. How you compare the files is up to you, but your analysis should be **quantitative** and ideally make good use of figures.

Sentinel-2 images

The Sentinel-2B satellite (Fig. 14) launched in 2017 (it was joined by the identical Sentinel-2C in 2024). Each of the Sentinel-2 satellites caries a single instrument, the "Multi-Spectral Imager". You can read the detailed of how the MSI works here, but in brief it uses a "push-broom" approach to scan a 290 km-wide swathe of the Earth's surface. This push-broom method is similar to how a photocopier or scanner works: as the satellite moves through its orbit, it scans a region of the earth beneath it and updates its detector register as it goes.

As the satellite orbits, it collects light reflected from a strip of the Earth's surface. This light is passed to either the Visible and Near-Infra-Red (VNIR) detectors or the Short Wave Infra-Red (SWIR) detectors, which measure the reflected intensity in one of 13 different spectral bands (filters). The filters have been chosen to be sensitive to specific surface properties of interest, for example showing reflected light from vegetation, or from clouds. You can find some discussion

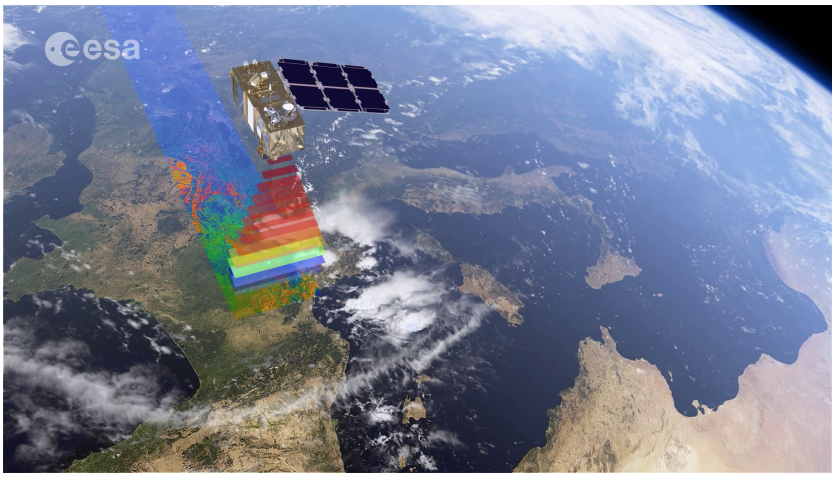


Figure 14: The Sentinel-2 satellite

of the filters used at this link. For ease of reference we have adapted the (very helpful) table from the linked site below.

Band	Resolution	Central Wavelength	Description
B1	60 m	443 nm	Ultra Blue (Coastal and Aerosol)
B2	10 m	490 nm	Blue
B3	10 m	560 nm	Green
B4	10 m	665 nm	Red
B5	20 m	705 nm	Vegetation Red Edge
B6	20 m	740 nm	Vegetation Red Edge
B7	20 m	783 nm	Vegetation Red Edge
B8	10 m	842 nm	Near Infrared (NIR)
B8a	20 m	865 nm	Narrow NIR
B9	60 m	940 nm	Water Vapour
B10	60 m	1375 nm	SWIR (Cirrus)
B11	20 m	1610 nm	SWIR
B12	20 m	2190 nm	SWIR

Table 1: Sentinel-2 Spectral Bands Specification

The European Space Agency provides a large number of tools to interact with Sentinel data, in particular through the Copernicus Browser. As Sentinel data are quite large, it is often easier to work with them in the cloud, rather than downloading them to a local machine. However, for this exercise we will work on a simple analysis of a downloaded subset of Sentinel data. On Brightspace you have been provided with "Level 2A" data products from Sentinel-2. These are bottom of atmosphere reflectances in cartographic geometry, in other words they have been corrected for the effects of atmospheric absorption, and mapped onto a flat surface. The resolution of these images will vary between 10 amd 60-m depending on the wavelength

in question.

The images have been taken of the Hunga Tonga–Hunga Ha'apai volcano that has erupted over the past few decades, forming new islands in the South Pacific. Your task is to use these, along with white light images from Google Earth to understand the changes in size and surface properties of this island.

Assignment

Exercises

1. Read in the images you have been provided with into numpy arrays. The `PIL` or `rasterio` packages can help with this.
2. Using the white light images, measure the area of the island as a function of time. A scale bar is provided in the corner of each image.
3. Within the Sentinel directory are several pre-generated products such as NDVI, NDWI etc. Using these, make a plot of the area covered in vegetation as a function of time.
4. You have also been provided with the images that are used to make NDVI, NDWI etc (these are the files that have the suffix `.raw`. Use these to make your own NDWI and NDVI. How do these compare to the ones you have been provided with?

To understand the Sentinel data products, you will need to consult the extensive online documentation

High energy detectors

Important — Radiation Safety

The radioactive sources in the lab are low intensity sources (provided they remain outside your body) but you must take certain sensible precautions:

1. Sources must only be used with the explicit consent of the lab instructor. The sources must be signed in and out, and only by the lab instructor.
2. The sources must be handled carefully. Plastic tweezers should be used when handling to protect the sources. Hold them at the top or the edge. Do not hold them in the center (either with tweezers or with fingers) — there is a risk that you will damage the protective slide.
3. There shall be no food or drink in the laboratory.

Gamma-ray astronomy is the study of astrophysical objects via the detection of their energetic photons, which can range from 10 keV to hundreds of GeV in energy. Energies below 10 keV are generally described as being x-rays.

In order to efficiently detect γ-ray emissions from astrophysical sources, the correct scintillator and/or detector combination needs to be used, as each detector is sensitive over a particular part of the electromagnetic spectrum, with a quantum efficiency that varies with respect to energy.

In this lab you will calibrate and characterize different γ-ray detectors. Each type has flown on past and present γ-ray astronomy missions. You will examine the behavior of these detectors and write a report discussing their performance and suitability for different space missions.

The experiments you do will follow the pre-flight tests performed on the detectors for the Fermi Burst Alert Telescope, documented in Bissaldi et al. (2009). You should read this paper, which can be found

on BrightSpace.

You will be assessed based on your code, which is to be submitted to GitHub, and your final report, which is to be no less than 5 pages and no more than 10 pages including tables and figures. Your final report should include both a presentation of your results and a discussion about the characterization of the detectors and their suitability for space missions.

Theory and background

γ-ray detectors work by converting the energy deposited by high-energy photons when they interact with the detector material into measurable electrical signals. To do this, they use the deposited energy to excite electrons within the crystal structure. These excited electrons generate an electrical signal, which is measured and digitized. Since the number of excited crystal electrons increases with the amount of energy deposited in a detector, the size of this response measures the energy deposited by the γ-ray.[21]

For each incoming photon, the output from the detector is digitized. This adds a count into the appropriate digital energy channel. Once many γ-rays have entered the detector and deposited (some or all of) their energy, the resulting spectrum consists of the number of counts in each channel.

The detectors you are required to investigate are:

- a Cadmium Telluride detector (CdTe);
- a Thallium-doped Sodium Iodide detector (NaI:Tl); and
- a Bismuth Germanate detector (BGI).

These represent two types of detector, which are distinguished by how they convert excited electrons into an electrical output. These are:

- Solid state detectors (CdTe and high-purity germanium, or HPGe) directly detect the energy absorbed from ionizing radiation as an electrical current generated when the high energy photon deposits energy and excites electrons into the conduction band of the material.
- Scintillator detectors (NaI and BGO) have crystals which absorb high-energy photons and re-emit low-energy photons at visible wavelengths. A photomultiplier (usually a photomultiplier tube [PMT]) is used to convert this light into an electrical signal that can be amplified and digitized.

Make sure that you understand the differences between the physics of these two detector types: they are fundamental to understanding

[21] While the energy deposited by each incoming photon is typically tens to hundreds of keV, the bands within the crystral structure of the detector crystal are typically on the order of a few eV. This means that the energy deposited by each high-energy γ-ray produces a macroscopically detectable number of excitations, and so a measurable detector response.

the similarities and differences between the resulting spectra. You will need to address this topic in your report and use it to inform you discussion.

Radioactive sources

You will use pre-calibrated radioactive sources to determine the spectral response of the lab detectors. We will use 4 radioisotopes in this lab. These radioisotopes consist of unstable nuclei, which decay over time via α, β, or γ-decay to a stable nucleus. The decay schemes for these radioisotopes are shown in Table 2.

Isotope	Decay Mode	Half-life $\tau_{1/2}$ / yr	Energy / keV	Emission Fraction
^{60}Co	β^-	5.2714(5)	1173.228(3)	0.9985(3)
			1332.492(4)	0.999826(6)
^{133}Ba	EC	10.537(3)	53.1622(6)	0.0214(3)
			79.6142(12)	0.0265(5)
			80.9979(11)	0.329(3)
			276.3989(12)	0.0716(5)
			302.8508(5)	0.1834(13)
			356.0129(7)	0.6205(19)
			383.8485(12)	0.0894(6)
^{137}Cs	α	30.09(11)	661.657(3)	0.8499(20)
^{241}Am	α	432.2(6)	26.3446(2)	0.024(3)
			33.1963(3)	0.00121(3)
			59.5409(1)	0.3578(9)

Table 2: Properties of nuclides used in this experiment, including decay modes, half-lives, and photon energies of major emission peaks. Data from appendix B of Gilmore et al. 2008

For decay by α emission, we may look at ^{241}Am as an example to help us understand the process. This has a half-life of 432 years, and decays to ^{237}Np by emitting an alpha particle, which is just $^4_2$He. In doing so, it populates excited levels in ^{237}Np, one of which is the $\frac{5}{2}^-$ level at an excitation energy of 59.5 keV. After 67 ns, this level decays to the ground state of ^{237}Np via emission of a 59.5 keV X-ray.

^{60}Co on the other hand has a 5 year half-life and decays by β^- emission to ^{60}Ni. This decay populates the 4^+ excited state in ^{60}Ni, which promptly undergoes a rapid cascade (on picosecond timescales) to the ground state in this stable nucleus, emitting two γ-rays with energies of 1173.2 keV and 1332.5 keV.

Table 2 contains some details about the principal emission lines that you will encounter inthe lab, including the nominal energy of the transition and the fraction of decays that result in emission of a photon of that energy. Note, this table does not contain all the

emission features seen with these isotopes. You would need to consult catalogs of reference spectra to correctly identify all the decay features, especially when trying to analyze lower-energy features in spectra from high-resolution detectors. One example of such a databse is the Gamma-ray Spectrometry Catalog, hosted by the Idaho National Laboratory.

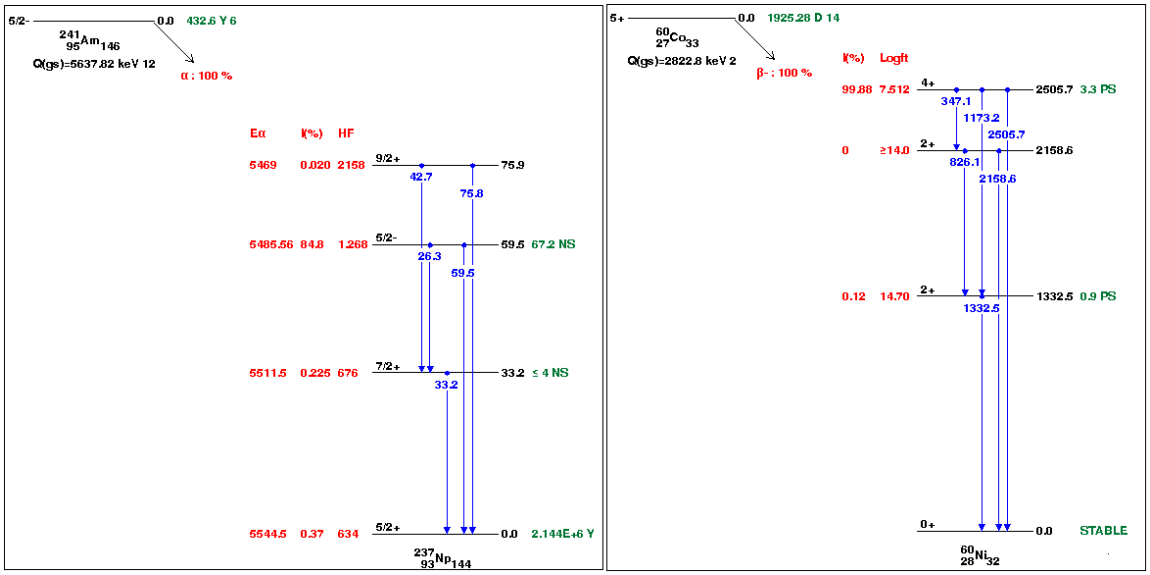


Figure 15: Left: the decay scheme of ^{241}Am to ^{237}Np showing the 60 keV transition. Right: The decay scheme of ^{60}Co to ^{60}Ni, showing the prompt cascade of the 1173 and 1332 keV transitions. Images are modified from Brookhaven National Laboratory.

Basics of Gamma-Ray Spectra

The radioactive source emit discrete energy lines. However, the energy spectra produced by the detectors are much more complex. High-energy photons interact with the detector by 3 principal methods:

1. the photoelectric effect;
2. Compton scattering; and
3. pair production.

These combine to produce complex spectra like that shown in Figure 16. The spectrum has been produced by a monochromatic source (that is, a source emitting photons of a single energy; namely 662 keV for ^{137}Cs). We might expect that this would produce a narrow peak corresponding to that emission line; however, while we can see a clear peak (termed "photopeak"), there are also features at lower energies caused by photons that have only *partially deposited* their inident energy in the detector. Partial deposition can occur if either

the γ-photon or the products of its interaction escape the detector after its first interaction.

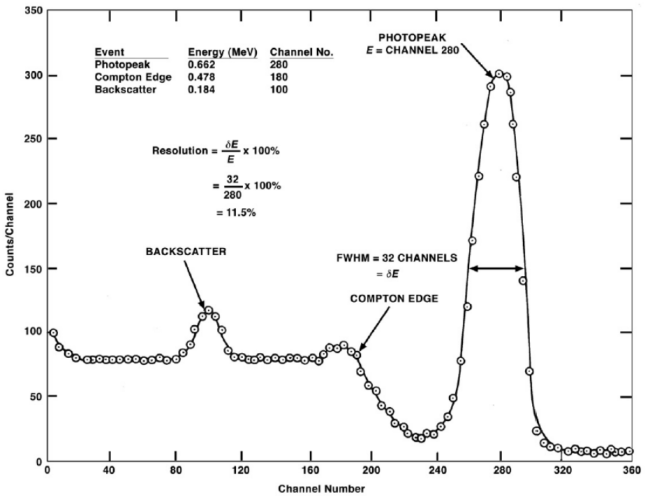


Figure 16: An energy spectrum showing the 662 keV photo-peak of ^{137}Cs as detected by a NaI:Tl detector. Figure originally from Oregon State University.

In this figure, we see the following features:

- the Compton feature, a continuous spectral feature caused by γ-rays scattering off electrons and then escaping the detector;
- the electron-position escape peaks, where pair production happens close enough to the edge of the detector that one or both of these particles escapes, carrying away its rest-energy; and
- X-ray escape peaks, where a γ-ray undergoes photoelectric absorption near the detector surface and the resulting photoelectron escapes, carrying away its kinetic energy and leaving behind just the binding energy taken to eject it.

In principle, if the source activity is high compared to the detector response and readout times, we may also see significant numbers of counts above the photopeak due to count pile-up. This happens when two photons enter the detector sufficiently close together in time so that the detector cannot distinguish between them. In practice, unless the source is very active or the detector takes a very long time to process each γ-ray event, pile-up happens so rarely that the spectrum falls off rapidly beyond the photopeak. You will need to discuss these processes in your final report.

Experiment

Problem statement

For this portion of the laboratory you will be tasked with calibrating and characterizing the performance of the detectors. This is done so that you can compare and contrast the detectors, commenting on similarities and differences, as well as strengths and weaknesses, and comment as well on the suitability of certain detectors for certain missions.

Because our spectral data is composed of channels and counts, you must first calibrate each detector to determine its energy response. To characterize the performance of a detector, you must determine:

1. the energy resolution as a function of energy;
2. the absolute and intrinsic efficiencies as functions of energy; and
3. the off-axis performance of the detector, e.g. efficiency as a function of angle.

You will need to develop an appropriate set of **data analysis pipelines**, which means that you will write a set of scripts that take inputs as spectral files and appropriate configuration information, and output the appropriately analyzed data products, including tables and graphs.

Data Collection

You will collect your data from the specified detectors, working in small groups and sharing this data with your team members. It is your responsibility as a group to ensure clear communication among yourselves, such as agreeing on a consistent approach to documenting data collection, naming conventions, and so on. Be aware that if any information is missed, there may not be time to repeat the data collection.

Data Extraction & Analysis

The spectra will be saved as either `.mca` (for the CdTe detector) or `.Spe` files (for NaI and BGO), which are plain ASCII text files. These can be opened with any text editor (such as Notepad on Windows or TextEdit on Mac), which means they can also be read with Python.

Within your pipeline, you will probably need to write Python functions that take the files, return the spectrum (and any important metadata) in a common format to pass to the next stage of the pipeline.[22]

[22] It would be wise to plan ahead in this regard, and create a clear "action plan" for processing your data.

Once the data is collected, you will need to analyze it by fitting models to the spectra. A best approach is to fit a parametric model to each of the photopeaks, as you can determine the parameters of that model (e.g. line position, width, and area) and use these results in your calibration and characterization.

Typically when dealing with spectra, our first approach is to fit a Gaussian line profile of the form:

$$f\left(x;\mu,\sigma^2,A\right) = \frac{A}{\sigma\sqrt{2\pi}}\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \tag{14}$$

While this is a good place to start, it is not going to completely describe both the continuum and the line; and in some cases, particularly where there are multiple poorly resolved decay channels, it may not be possible to fit the spectral line components separately.

As such your pipeline should be able to handle compound models that are more complex than a simple Gaussian. You may choose to fit a Gaussian superimposed on a low-order polynomial as a first attempt. This would allow you to extract your Gaussian parameters as well as the "nuisance" background parameters which you can likely ignore.

It is also advisable to attempt to fit one line at a time. It may be tempting to fit the whole spectrum, but the actual shape is incredibly complicated and requires considerable physical modeling, which is beyond the scope of our tasks here. Instead, identify regions of interest within which you can model the spectrum using a simple Gaussian model.

You may also find it necessary to subtract the background count rate first before attempting to fit the spectrum. This is particularly important for low-activity sources where the shape of the spectrum can be "muddied" by the background. Examining your data prior to writing any routine like this is your best approach.

Finally, you should consider whether you want to work with raw counts or a count rate. It may seem unimportant, but it can matter greatly later on, such as when you are trying to subtract a background.

It hopefully goes without saying that all fitted parameters should be reported with their respective uncertainties, and these uncertainties much be propagated through to any derived quantities. If you are using the `curve_fit` function from the `scipy` module, you will want to extract the covariance matrix from each fit to help you here.[23]

[23] Ideally, uncertainties would be placed on the data themselves as well; though this is less important here. Poisson statistics mean the uncertainty is the square-root of the number of counts, but this can backfire spectacularly if the number of counts is below 20; even more so if the count rate and background rate are comparable in size.

Calibrating the Detector

Once you have collected your spectra, you will need to work out how to relate the digital channels to the absolute energies. You know

the energies of the photopeaks, and by fitting models to your spectra you can determine the channel containing the centroid of your photopeak. With these two pieces you can plot the channel-energy pairs and fit a calibration model to find the conversion factor between energy and channel.

The resulting plot, showing how the photopeak channel number varies with incident photopeak energy, is your calibration curve. The importance of this step cannot be understated, especially when attempting to identify γ-ray transitions in unknown sources.

Detectors are most useful if their energy calibration is linear, in which case the calibration curve has the form:

$$E = c_1 \times \text{channel} + c_0 \tag{15}$$

This should of course be tested (not assumed!) by fitting a higher order polynomial and checking to see if the higher coefficients are consistent with 0. For some detectors, you should consider which sources to use for this calibration depending on the energy response of that detector.

Characterizing the Detector

Source Activity

The activities of the laboratory sources can be determined based on their measured activity at a fixed data. You can use the half-lives from Table 2 and the time elapsed since the source calibration to calculate the present activity. You will need to note which set of sources you are using; this can be done by simply checking the label on the box containing the sources (e.g. "upstairs" or "downstairs") to ensure the correct measurement date.

Energy Resolution

The energy resolution measures how well a detector can resolve or separate two peaks that are close together in energy. The resolution of a photopeak centered at some energy E is:

$$R := \frac{\delta E}{E} \tag{16}$$

where δE is the full-width at half maximum (FWHM) of the peak.

This should be calculated as a function of energy, using the fits to the photopeaks in the on-axis spectra. You should include the main peaks from each of the isotopes, as well as any other peaks that can be clearly identified. Be careful however when attempting to measure the energy resolution for the photopeaks in ^{133}Ba: the

complex emission structure around 300 keV can produce strange results, especially in detectors with lower resolution.

The energy resolution R approximately varies with energy as:

$$R^2 = \left(\frac{\delta E}{E}\right)^2 = \alpha E^{-2} + bE^{-1} + c \tag{17}$$

which should be fit to your resolution vs energy plots for each detector (see Bissaldi et al. [2009] for more details about this relationship). The power-law behavior of this equation will be hard to see and interpret on a linearly scaled graph, so it is advisable to plot the energy-resolution plot on logarithmic axes. It may be easier to fit this equation if it is expressed in terms of $(\delta E)^2$ rather than R^2.

Efficiency

The efficiency of a detector is related to the fraction of events that the detector detects. For our purposes, we are concerned with the *absolute* or *full peak efficiency*, and the *intrinsic efficiency*.

The absolute efficiency is the fraction of *all* emitted photons that are detected:

$$\epsilon = \frac{\text{count rate}}{\text{source activity}} \tag{18}$$

This depends both on the detector performance and on the relative geometries of the source aand the detector.

The intrinsic efficiency, as the name implies, should be an intrinsic property of the detector material. It is the fraction of photons that pass through the detector and are detected:

$$\epsilon = \frac{\text{count rate}}{\text{rate that photon pass through the detector}} \tag{19}$$

The two efficiencies are related by a geometry factor G, accounting for the cross section of the detector as a fraction of the full sphere. You will need to devise appropriately justified models to calculate this, for which you will need to record appropriate parameters of the experimental setup. Make sure your team agrees on a procedure for doing this during your data collection.

The intrinsic peak efficiency as a function of energy can be approximately described by a low-order polynomial[24] in log-efficiency vs log-energy:

$$\ln \epsilon_p = a + b \ln E + c(\ln E)^2 \tag{20}$$

You should fit this equation to your data and display it on your graph. The logarithmic character of this function should clearly suggest that the plot should be given on logarithmically scaled axes. You

[24] This relationship is empirical rather than theoretical, but it allows us to describe a simple power-law relationship across a wide energy range from 10-1000s of keV, with a "turnover" at low energy. We can explain the high-energy shape by arguing that the detector is less effective at stopping higher energy photons. The low-energy turnover occurs due to the slight shielding effect of the detector housing to lower energy photons.

may find it easier to fit a polynomial to $\log \epsilon_p$ vs $\log E$, rather than working with the full equation.

Off-Axis Response

The relative response of a given detector at various angles is very important in γ-ray astronomy, as the photons from astrophysical sources are quite unlikely to be centered on-axis with the front of the detector in the moving spacecraft. As such, you will need to examine the response of the detector as a function of source angle by collecting appropriate spectra at appropriate angles. It is your responsibility to determine which parameters of the detector are likely to vary as a function of angle, and which ones are unlikely to change. You can explain your reasoning (briefly) in your report.

When plotting these parameters as a function of angle, it may be helpful to normalize the plots to their on-axis values. Doing so may make it easier to see off-axis variation, particularly when trying to compare different detectors.

Primary Summary

Your tasks are to provide (for each detector):

- code to produce a calibration curve (channel to wavelength conversion);
- code to determine the energy resolution (FWHM of photo-peaks) and create a plot as a function of energy;
- code to determine absolute and intrinsic efficiencies, plotted as a function of energy
- code to characterize off-axis response (change in FWHM and peak height) as a function of angle.

These codes should be part of a working data analysis pipeline, so it is important to ensure they all communicate well with each other and function as part of a whole.

Comparing results

Once you have completed your coding and analysis, you will have one final task: comparing your results. This additionally assumes that you have submitted your (functioning!) code to GitHub. Each group will take the results from all other groups and produce a resolution vs energy plot from all of the data. It will be up to the groups and the class to coordinate on how to make this happen.

Primary Summary

Your final task is to provide a brief (10 pg)

- Report - brief overview comparing your results to others
- Discuss which detectors are useful for which type of mission

Addendum: Building a Pipeline

The easiest way to build a data analysis pipeline is to first get working components for each section of your analysis, testing these individually (in Jupyter notebooks, for example) before putting them all together. A good approach would be to create separate Python files for each of the important pieces of your workflow: for example, your calibration code may be in `calibration.py`, your resolution code in `resolution.py`, and so on. You may even take it further and build your pipeline with classes. As discussed in the Python section, you can exploit relative imports here and use a module like `sys` or `argparse` to control command-line arguments.

It is not advisable to write a "do it all at once" pipeline, but you should also resist the urge to allow a user too many options. Sometimes it is helpful to think in terms of command-line arguments to inform your workflow. For example, we may construct a pipeline using the `argparse` module where the user can interact with it like so:

```
$ python pipeline.py --detector BGO
    --calibrate
```

Our logic here might be that we always begin with calling `pipeline.py` and pass specific arguments to it; in this case there are the arguments `--detector <DETECTOR_NAME>` and `--calibrate`. As such, our pipeline has a clear and simple interface: it needs a detector and a function and performs that task only. Of course, we may need to save the results of the calibration before running any of the other functions, but that is easy to implement.[25]

It is also important to note that you should avoid under all circumstances using fixed path names, especially if others will be running your code. While using GitHub guarantees that all team members working on the code will be using the same directory structure, the team members may have very different operating systems and thus very different path names. You can circumvent this issue with either the `os` module or the `pathlib` module, and by using relative paths rather than absolute paths.

[25] Another, perhaps more complicated option, is to have your pipeline load a user-defined configuration file and parse all the necessary data from that. A few popular and modern config file formats are the `.JSON` and `.YAML` formats.

AMSATs

Introduction to AMSATs

CubeSats are a type of small "nano-satellite" designed for low-earth orbit. Initially intended simply to assist students design, manufacture, test, and deploy their own small spacecraft, the CubeSat eventually became a standardized form used by amateurs, professionals, and commercial enterprises around the world. An especially attractive aspect of these small devices is that their affordability can justify the risks associated with experiments founded on "weaker" underlying theory. Additionally, their small form factor (each device is a 10 cm cube) and low weight ($m \sim 2$ kg) allows them to "piggyback" on larger missions with minimal risk to the larger payloads.

The standard size of a CubeSat is 1U (or 1 unit), though many larger 3U CubeSats have been designed. It is also possible to combine multiple 1U or 3U CubeSats into larger composite CubeSats if desired. While CubeSat standards do not specify electronics designs or communication protocols, we typically see the hardware designed with commercial off-the-shelf (COTS) components which range in complexity from simple Raspberry Pi devices to more specialized, custom-designed payload boards built to withstand irradiated environments. Perhaps the most (locally!) exciting example of a functioning CubeSat is EIRSAT-1, a 2U device developed by none other than the University College Dublin and launched in December, 2023, becoming Ireland's first satellite.

You should, of course, read all about it on the official EIRSAT website.

In our laboratory, we will be using the CubeSat Simulator (CubeSatSim for short), a project funded by the Radio Amateur Satellite Company AMSAT. The AMSAT is an open-source project (with software maintained on the CubeSatSim GitHub page) featuring three custom printed circuit boards (PCBs) which control the Main, Battery, and Solar panel busses. The primary controller for this device is a Raspberry Pi: a small, Linux-based single-board computer popular in academic and hobby settings alike. The AMSAT contains several sensors and connections for solar panels, and additionally has a radio feature that can broadcast real-time telemetry to a specified ground

While strictly speaking this device is the AMSAT CubeSatSim, we will simply call it the AMSAT for brevity's sake.

station.

In this portion of our course you will be working with these AMSAT devices to simulate real-life interaction with nanosatellite missions. We will discuss communication over `ssh` and Wi-Fi connections, and you will learn how to extract, decode, parse, and analyze real-time telemetry. Additionally you will gain hands-on experience preparing, 3D-printing, and assembling structural components for the AMSAT housing. The bulk of your work will be done in Jupyter Notebooks, and at the end of the section you will be expected to produce a short (2 pages) summary of what you did, including key plots or tables from your experiment and a critical commentary on your own work.

Assembling AMSATS

Three of the four housing pieces for the AMSAT will be provided for you, and your group will be required to print the final piece and finish the housing assembly. The 3D printed design can be found on the CubeSatSim GitHub (linked above) or on Thingiverse as an `.STL` file. Your task will be to locate this file, download the appropriate software to open and process this file, and print it. The ready-to-print file can be placed on an SDCard (which will be provided in the lab) and inserted into one of the Bambu Mini 3D printers available in the lab.

It is important to ensure you have printed the correct piece! You will need to do some research to find your AMSAT version (hint: it's printed on the PCBs) and search the CubeSatSim Wiki to find the appropriate `.STL` file for that version. It is strongly recommended that you familiarize yourself with this Wiki as it contains full documentation of the AMSAT device and plenty of photos at various stages of assembly. We will provide the nuts and bolts and tools for the assembly. A fully-assembled AMSAT can be seen in Figure 17.

Connecting to AMSATs

SSH & WiFi

Once you have assembled the AMSAT housing, we can now connect to the device. However, to connect to our AMSAT we have to do some configuration. Recall that when using platforms like GitHub we had the option use the secure shell or ssh protocol to clone a repo to our local machine. A more powerful use of the ssh protocol is that it can actually let us directly connect to a remote host and issue commands to the host as though it was our own local machine. In

Figure 17: A fully-assembled AMSAT, displaying the housing, a side-view of the PCB stack, and one of the side-mounted solar panels. A small RBF ("remove before flight") pin can also be seen, which, when removed, triggers the RF mode and begins broadcasting telemetry packets over radio.

order to do this though we need to configure something called "ssh keys" on our local machine. On Windows machines this is not a very straight-forward process, but in our lab we will be using Linux machines with pre-configured keys so you will not need to be concerned about the details of this.[26] We will assist you with the login details for these machines in the lab.

[26] If you *are* interested in the details, see the GitHub documentation about creating new ssh keys.

Just like with any other real-world communication, we need to know the *address* of the machine we want to reach, and more importantly it needs to be a *reachable* machine. Most often the address is an IP address or a `<HOSTNAME>.local` address. When you open a terminal in Linux or Mac, the user and hostname are typically shown automatically; on Windows machines we can view the hostname in the System Settings. For the machine to be reachable it needs to either be on a local network or some remote network that is configured to allow external access. For our purposes, we will be using the AMSAT on a local network using either a wired USB connection or a Wi-Fi connection.

USB Ethernet connection

The first thing we need to do is use a micro-USB cable that allows data transmission as well as provides power. We will provide these in the lab, as well a cable tester you can use if you have any doubts. The micro-end of the cable goes into the middle input port on the Raspberry Pi (on the bottom of your AMSAT device), and you can plug the other end of your cable into the lab computer. When the AMSAT is powered on, you will see a blue LED turn on. This may take a few minutes to appear.

On the lab computer, the toolbar at the top right of your screen should show an internet connection icon; select this icon. In this window, if you see "PCI Ethernet Connected" then this tells you that your lab computer has an ethernet connection to the UCD network, so we want to leave that one alone. The AMSAT will show up under the "USB Ethernet" connection, and if the AMSAT is fully powered on you will see either "USB Ethernet Connected" or "USB Ethernet Connecting". Select the "cog" icon next to the UBS Ethernet option, alternatively, select the"Settings" or "Wired Settings" option, as shown in Figure 18. If you need further assistance finding these options (some version of Linux may display these things differently) let your instructor know. This will open the Network Settings dialogue.

By default, Linux will try to assign an IP address to your AMSAT automatically using DHCP. We do not want this. In the Network Settings window, select the wheel next to the USB Ethernet option and click the IPv4 tab. Make sure that the "Link-Local Only" option is selected in the IPv4 Method box (see Figure 19). Click the IPv6 tab and make sure the IPv6 method is disabled. Save your connection settings and close the window.

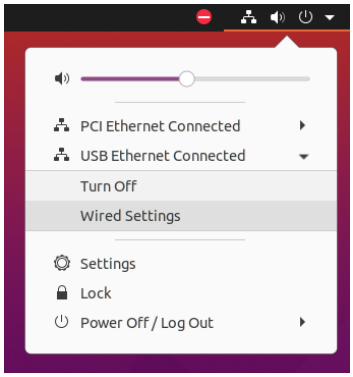


Figure 18: The Linux network dialogue showing the different connections and options available.

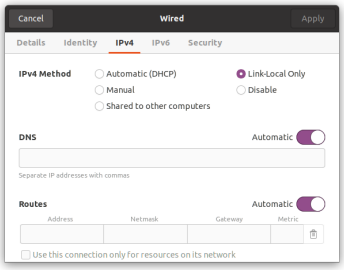


Figure 19: The Wired Connection settings window. The IPv4 method is Link-Local Only, and IPv6 is disabled.

Finding the address

Now that the AMSAT is configured as a link-local USB ethernet device, we can connect to it through the ssh protocol. Typically the address of a remote device has the format:

```
# IP Address format
$ <USERNAME>@127.0.0.1
# .local address
$ <USERNAME>@<HOSTNAME>.local
```

For the AMSATs, the default user and host names are `pi` and `cubesatsim<N>.local`, where `<N>` is the device number (see the sticker on your AMSAT box) and the default password is `raspberry`. It is generally easier to know the hostname rather than the IP address, especially if the device is not in our local network. With this knowledge we can connect to the device pretty easily. You can click the terminal application icon on the left panel (it looks like >_) or type `ctrl+alt+t` to open it with the keyboard. For this example I am using `cubesatsim1`, so in the terminal I will type:

```
$ ssh -p 22 pi@cubesatsim1.local
```

The `-p 22` tells the ssh client to connect over port number 22; this is the default port number used by ssh but we include it here just to be safe.

If the connection was successful and this is the first time you have connected to the device, you will be shown a warning message like the one below with a key fingerprint and asked if you want to continue with the connection. We can type `yes` because we know this is

a trusted device, and then enter the default password for the AMSAT as noted above.

```
The authenticity of host 'cubesatsim1.local (<IP_ADDRESS>) can't be established
    .
ECDSA key fingerprint is SHA256:<SOME_STRING>.
Are you sure you want to continue connecting (yes/no.[fingerprint])?
```

Communicating with the AMSAT

All going well you are now connected to the AMSAT and your terminal should show `pi@cubesatsim<N>:~$`. Since this is a terminal interface and not the more familiar graphical user interface with clickable folders, we have to use text commands to navigate and display information. You have seen some of these already with the Git commands; you are encouraged to explore this GitHub page which has a nice summary of the most common commands. Use the `ls` and `cd` commands to navigate around a bit and get more accustomed to the AMSAT directory structure.

WiFi Connection

As you will have noticed, connecting over a USB can be cumbersome and, at times, a bit temperamental.[27] It is also quite limiting for any experiments involving movement. Thankfully each AMSAT device is built with a Raspberry Pi Zero W, which is Wi-Fi capable, but we need to figure out a way to connect without using a monitor.

[27] It is especially fun if we are unable to find a USB cable to that transmits both data *and* power.

There are a few options here. The first and easiest option is to simply create a file called `wpa_supplicant.conf`, which is simply a configuration file that we place on the Raspberry Pi that will be used to automatically log-in to the WiFi network specified. We can create this file by simply opening up a text editor (like Notepad on Windows). This file must have the following contents:

```
ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev
country=IE
update_config=1

network={
   ssid="NETWORK_NAME"
   psk="PASSWORD"
   key_mgmt=WPA-PSK
}
```

where were replace `ssid` and `psk` with the appropriate values. Once this is created, we can place it on the Raspberry Pi SD card. After making sure your AMSAT is safely powered down and disconnected, remove the SD card, place it into a USB card reader, and plug it into one of the lab machines. If using the Linux machines (recommended) you should see the SD card mounted on the left hand panel; if you cannot find this ask one of the demonstrators. We want to open the `boot` folder and move the `wpa_supplicant.conf` file there. Safely eject the SD card and re-insert it into the AMSAT.

If all goes well, when the AMSAT boots up it will automatically connect to the specified network. You can then connect your laptop to the same network (this is important) and access the AMSAT through wireless SSH.

The second option is to SSH into the AMSAT using the USB/Ethernet method and access the Raspberry Pi configuration tool:

```
$ sudo raspi-config
```

This will bring up a window as seen in Figure 20. As this is a *terminal* user interface (as opposed to graphical) we navigate using the keyboard, not the mouse. Under "System Options" we can configure the Wireless LAN, providing the network name (SSID) and password. Alternatively this can be done in one line from the terminal:[28]

[28] For more information see the Raspberry Pi documentation.

```
sudo raspi-config nonint do_wifi_ssid_passphrase
    SSID PASSWORD
```

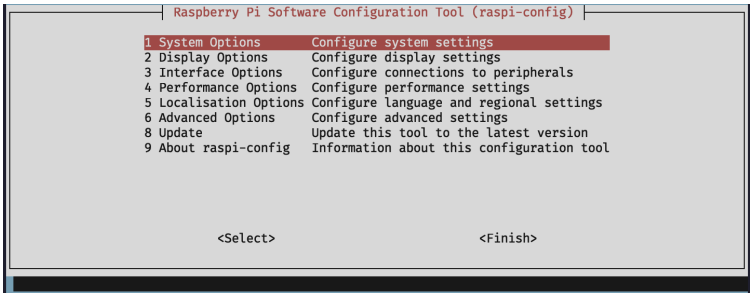


Figure 20: A screenshot of the `raspi-config` interface. From here you can adjust several features of the Raspberry Pi.

The third option also requires that we establish an SSH connection first. This method also requires that we create the `wpa_supplicant.conf` file, and move it to the appropriate directory. We can create this file exactly as above (i.e. using a laptop or one of the lab computers) and use `sftp` to transfer the file to the AMSAT (more on this method below), or we can create the file on the AMSAT directly and move it there. If you choose to create this on the AMSAT, you can type in:

```
$ nano wpa_supplicant.conf
```

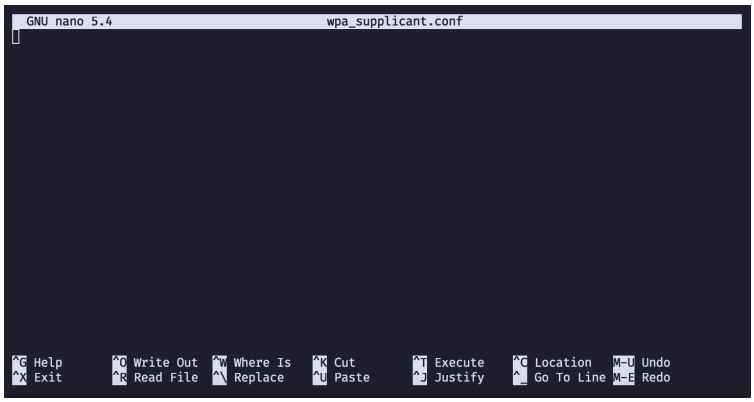


Figure 21: The nano text editor interface.

This will open the nano text editor as seen in Figure 21. When you have typed out the contents as above, press ctrl+o, then Enter to confirm the name of the file, and ctrl+x to exit the editor. Now we move the file to the appropriate directory:

```
$ sudo mv wpa_supplicant.conf /etc/wpa_supplicant/
    wpa_supplicant.conf
```

The sudo is important here.

The latter two options are probably the most complicated; it would be easiest to just create the file on a local computer and move it to the AMSAT's SD card.

Reading sensors

Because the AMSAT is ultimately a simulator, we can use it to gain familiarity with same of the basic functionalities of typical satellite subsystems. There are three sensor modules we will be working with on these devices.

- the **BME280** – measures temperature, pressure, and humidity;
- the **INA219** – a power monitor which measures voltage and current;
- the **MPU6050** – a combined accelerometer and gyroscope which measures motion in 6 axes

Provided with the AMSAT are two programs that allow us to access these measurements, and we will use a combination of Python and Linux commands to extract, save, and analyze the output. Before we explore how to use these programs, we should first distinguish between two important types of programs, and understand how to run them in on Unix-like systems.

Pre-compiled binaries (like telem and cubesatsim) are standalone "executables" or "binaries" – that is, they are ready to run on

their own without any need for an interpreter to translate the code into a language the computer can understand. These can be run simply with:

```
$ ./PROG_NAME
```

The "dot-slash" (`./`) is very important here; see the Linux Information Project page for detailed discussion.

Scripts (like the Python files you will write) on the other hand are not stand-alone and these *do* require an interpreter – for example, with a Python script the computer needs to use the Python interpreter in order to read and understand our code. These can be run with:

```
$ python SCRIPT_NAME.py
```

Capturing Output

Before we discuss how to acquire data, we need to understand how computer programs communicate with their environments. This communication occurs over channels we call "standard streams", and there are three of these: **standard input** (`stdin`), **standard output** (`stdout`), and **standard error** (`stderr`). A visual schematic is shown in Figure 22. We will spare the longer discussion on exactly what these things are, but for our purposes it's good enough to understand that on Unix-based machines the communication streams can be redirected and captured via the command line.

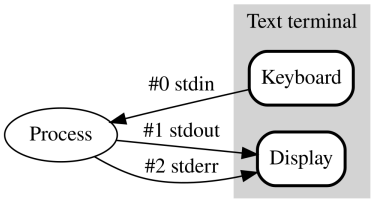


Figure 22: A schematic of standard communication streams. you can read more about standard input and standard output if you are interested. Figure public domain courtesy of Wikipedia.

If I want to capture all of the output from a program and save it to a file, the most efficient way to do it is to tell the program to direct its output streams to a file. We can do this easily:

```
$ ./PROG_NAME > FILENAME
```

The `>` symbol tells the program to send the `stdout` stream to `FILENAME`. Take care with this though because if `FILENAME` already exists, this will completely overwrite everything in that file. If we want to append to an already existing file, we use the » operator:

```
$ ./PROG_NAME >> FILENAME
```

If the program prints both the `stdout` and `stderr` streams, we might want to capture both. We can do this in one of two ways: redirect both streams to the same file, or redirect each stream to its own file.

```
# Send both streams to one file
$ ./PROGNAME &> FILE
# Send each stream to separate files
$ ./PROGNAME > FILE1 2> FILE2
```

Note that the files to which you save your output will be on the AMSAT, not your lab machine. To move them to the lab machine we can use the SSH file transfer protocol or `sftp`. Similar to the `ssh`

command we need to note the address and location of the file we want on the remote machine, and the location of where we want the move the file on our local machine. The syntax is simple:

```
$ sftp USER@REMOTE:PATH/TO/SOURCE /PATH/TO/
    DESTINATION
```

Let us assume we saved the output of `cubesatsim` to a file called `output.txt` in the `CubeSatSim` directory, and we want to move this file to the `Documents/AMSAT/` folder on our lab computer.

```
$ sftp pi@cubesatsim1.local:CubeSatSim/output.txt
    ~/Documents/AMSAT/output.txt
```

If we have typed everything correctly we will be prompted for the AMSAT password and the file will be copied to our `AMSAT` folder. Try this yourself, replacing `output.txt` with whatever filename you chose when using the `./cubesatsim &> FILE` command.

The next issue we face is parsing this data. The AMSAT programs produce (somewhat) human-readable data, but we will find that we cannot simply load our files into, say, Excel, because they are not in a machine-readable (i.e. structured data) format. The solution to this is straight-forward but a bit tedious: at its most simple, we search for a pattern in a string, e.g.

```
if "Some string" in line_of_text:
    # operate on that line
```

and carry on. A more advanced but rigorous approach is to use **regex** or "regular expression" matching to search the data and extract the pieces that we need.[29] This is essentially like an automated `ctrl+f` in programming form. Whichever method we opt to use, we need to understand how the data is presented, hence the importance of examining the output of the programs before we develop a plan to parse it. We will cover this in more detail in the AMSAT practicals.

[29] The Python `re` module is excellent for this; however it is a more esoteric approach than the simpler string method.

Exercises

In our AMSAT practicals, we will be doing the bulk of our work in Jupyter Notebooks. On each lab machine we have provided a notebook which will provide you with an introduction to the AMSAT and how to interface with it using Python; a similar notebook can be found on BrightSpaces. You will additionally perform the following experiments:

- Placing the AMSAT on a turntable, record the voltage produced by the solar panels as a function of time. Exporting this data to a Jupyter notebook, determine the angular rotation rate of the AMSAT. The best way of doing this will be through a Fourier analysis of the voltage data; scipy and astropy contain many useful libraries for this.
- Determine the angular acceleration using the onboard accelerometer. How does this compare to what you indirectly infer from the solar panel voltages, and what can you say about the accuracy of the two methods, and the tolerance of the turntables?
- How does the degradation of solar panels affect the output voltage? You can explore this by covering part of the panels with paper and tape (do not apply tape directly to the panel surface) or alternatively by carefully coating one of the panels with chalk dust. How does the voltage and current change with fraction of the panel that is available?
- When you query a sensor over ssh (perhaps through a Jupyter notebook) it will take a finite time to execute. What is the delay introduced by the groundstation (i.e. your computer), compared to on-board processing? Is this constant, or does this change if the AMSAT is trying to do several things at once? You can explore this with the `time` command either from the terminal or through Python.

Each of the exercises above should be answered with a one page write-up, including where necessary any plots and figures. Focus on results and discussion rather than method or background.